

机器视觉技术

刘晓芳 孙明竹

liuxiaofang@nankai.edu.cn

sunmz@nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

Nankai University

©孙明竹

第三部分 深度学习图像处理

Deep Learning for Image Processing

➤ 讲授内容

第一部分 绪论

- 机器视觉的概念
- 程序设计基础

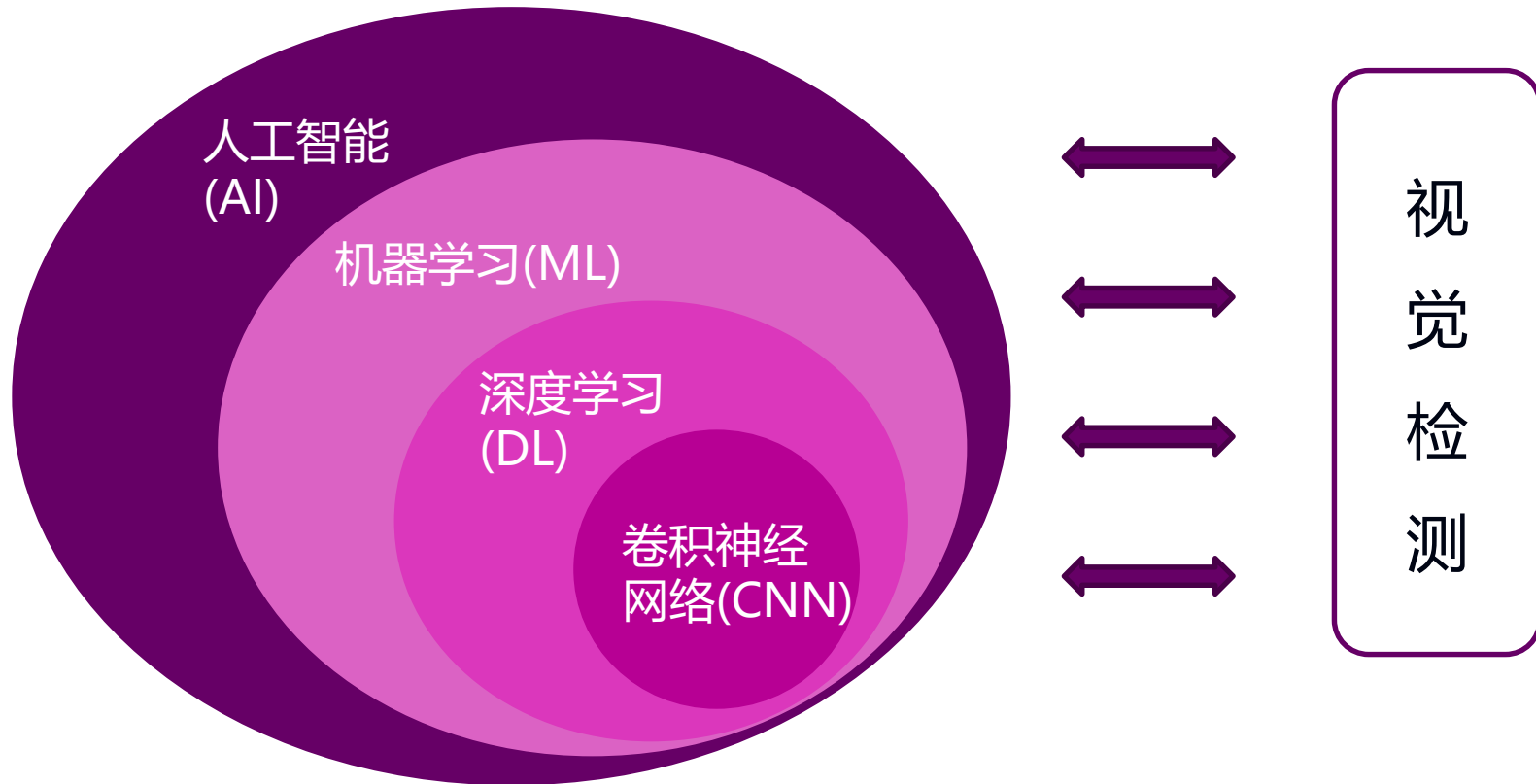
第二部分 图像处理基础

- 数字图像的概念与性质
- 图像预处理
- 二值图像处理
- 传统特征检测

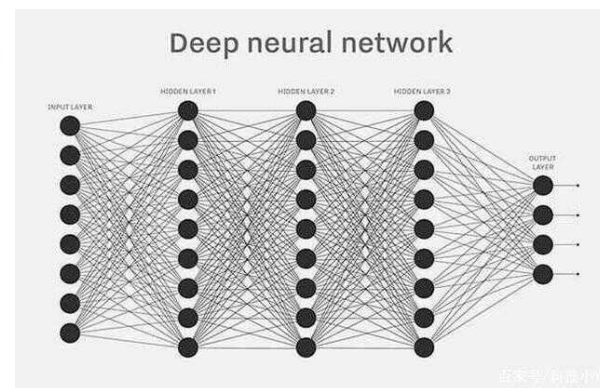
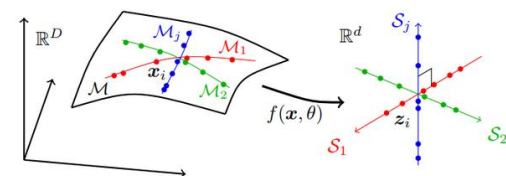
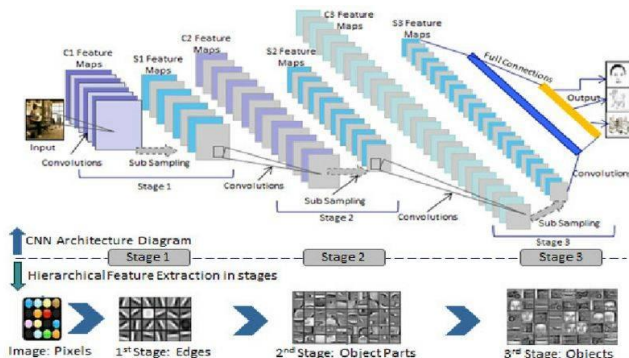
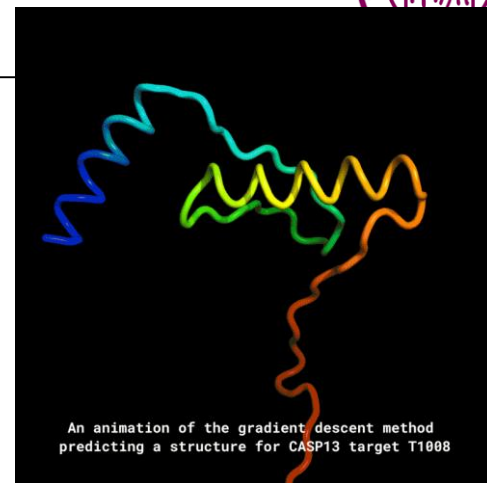
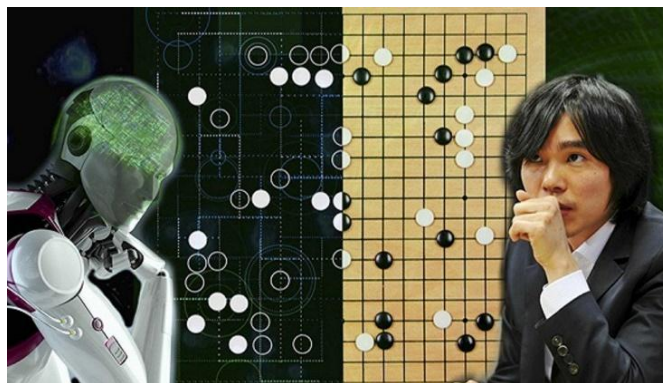
第三部分 深度学习图像处理

- 卷积神经网络
- 图像分类
- 目标检测与图像分割

从人工智能到卷积神经网络



人工智能的发展



人工智能的发展

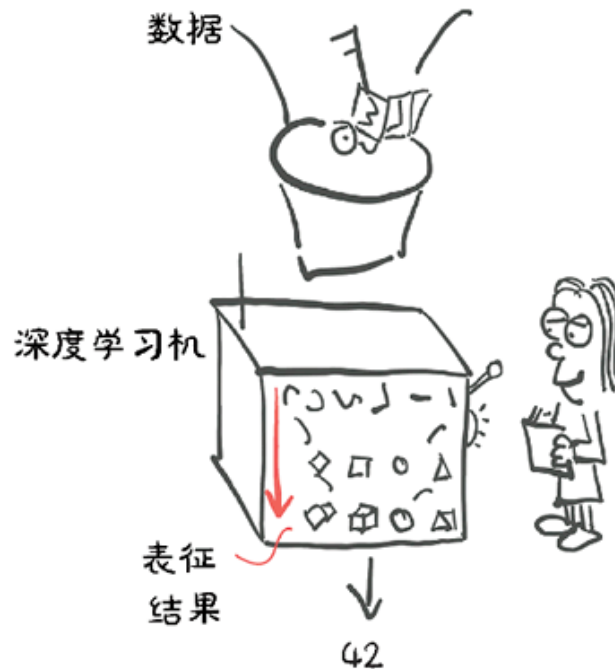
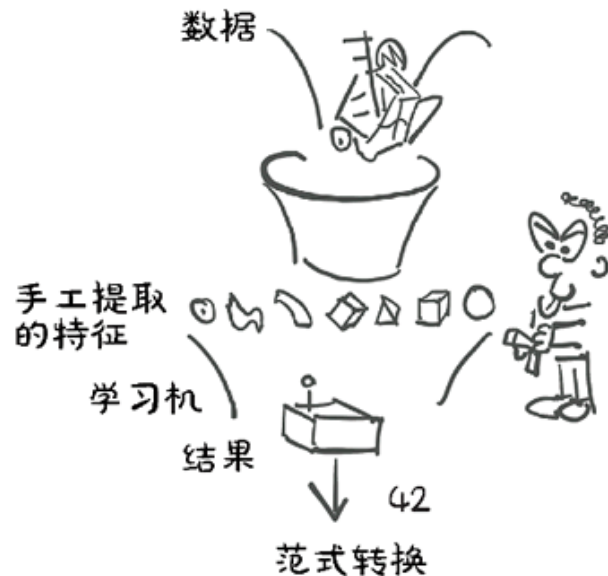


Midjourney V7绘画作品

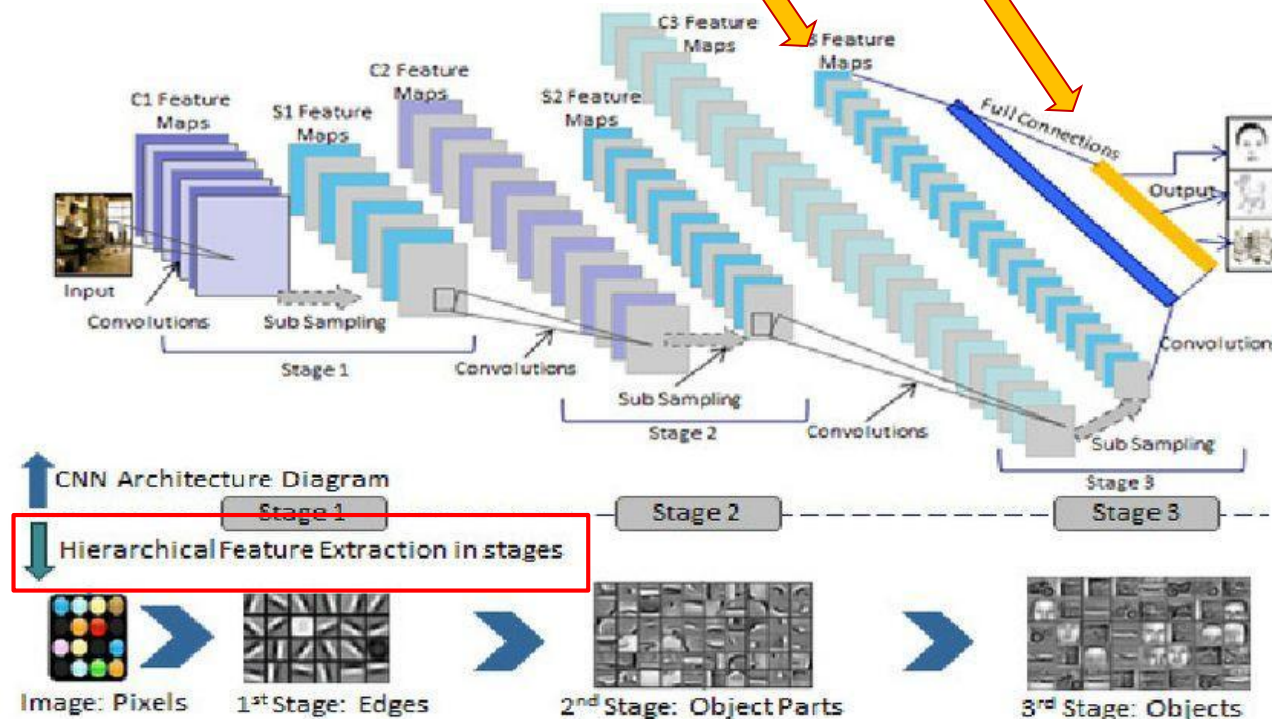


CSDN @阿噜噜小栈

传统方法与卷积神经网络



传统方法与卷积神经网络



传统方法

卷积网络

主要内容

- **第六章 卷积神经网络**
 - 6.1 深度前馈网络
 - 6.2 深度前馈网络架构
 - 6.3 卷积神经网络
 - 6.4 深度学习框架与视觉应用
 - 6.5 深度网络中的优化
 - 6.6 卷积神经网络的神经科学基础
 - 6.7 实践
- **第七章 图像分类**
- **第八章 目标检测与图像分割**

6.1 深度前馈网络

➤ 举例：学习XOR函数

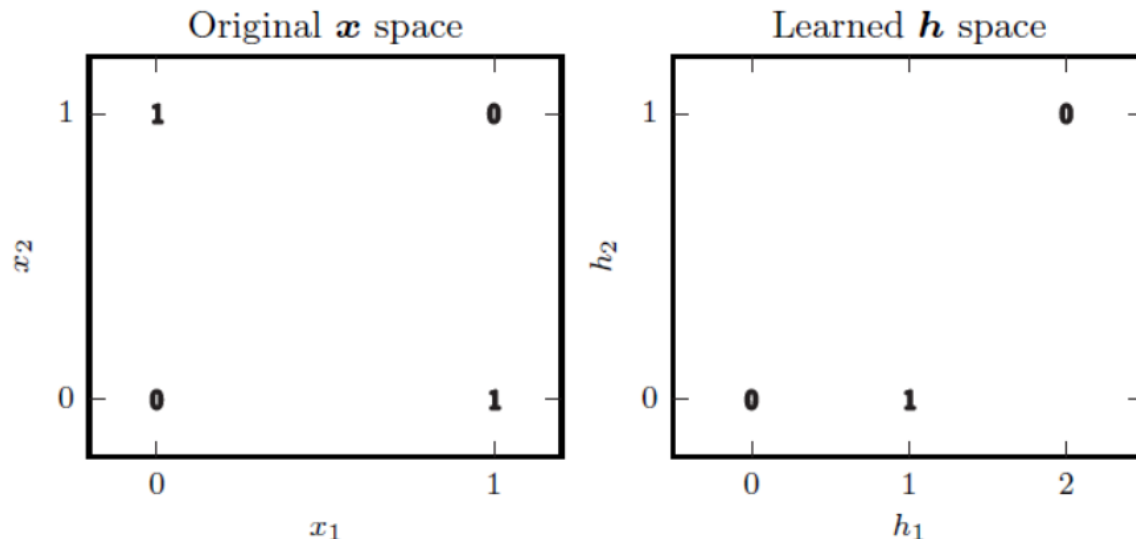
- XOR函数（“异或”逻辑）是两个二进制值 x_1 和 x_2 的运算。当这些二进制值中恰好有一个为1时，XOR 函数返回值为1。其余情况下返回值为0。
- 目标函数 $f(x)$ ：模型给出了一个函数 $y = f(x; \theta)$ 并且学习算法会不断调整参数 θ 来使得 f 尽可能接近 f
- 训练数据 $X = [x_1, x_2]^T = \{[0, 0]^T, [0, 1]^T, [1, 0]^T, [1, 1]^T\}$
- 评估函数均方误差(MSE)损失函数

$$J(\theta) = \frac{1}{4} \sum_{x \in X} (f^*(x) - f(x; \theta))^2$$

6.1 深度前馈网络：学习XOR函数

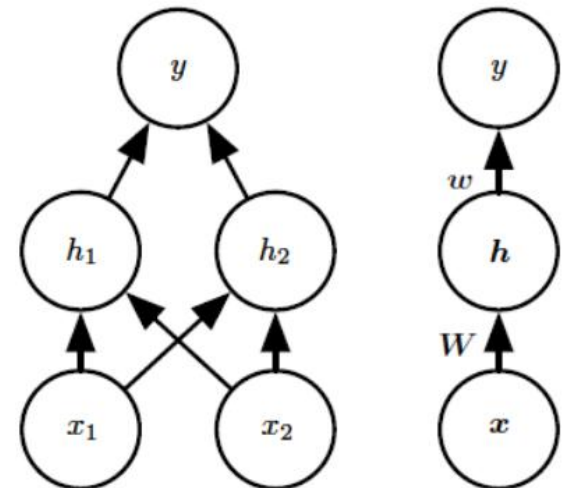
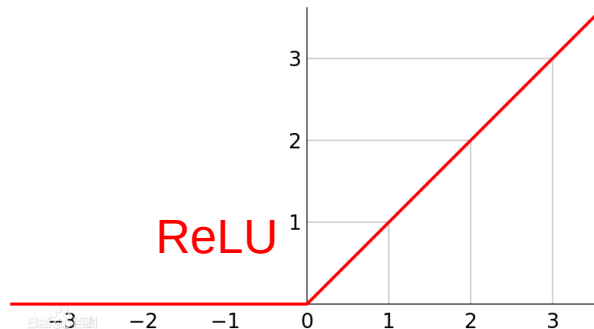
➤ 选择模型 $f(x; \theta)$

- 线性模型 $f(x; w, b) = x^T w + b$ ，找最小化 $J(\theta)$ 的 w 和 b ，得到： $w = [w_1, w_2]^T = [0, 0]^T, b = 0.5$
- 使用一个模型来学习一个不同的特征空间，在这个空间上线性模型能够表示这个解



6.1 深度前馈网络：学习XOR函数

- 前馈网络：有一层隐藏层并且隐藏层中包含两个单元，输出层是线性回归模型
 - $h = f^{(1)}(x; W, c)$ 计算得到的隐藏单元的向量 h
 - $y = f^{(2)}(h; w, b)$ 得到近似结果 y
- 完整模型： $f(x; W, c, w, b) = f^{(2)}(f^{(1)}(x))$
 - $h = g(W^T x + c)$
 - $g(z) = \max\{0, z\}$ 是激活函数



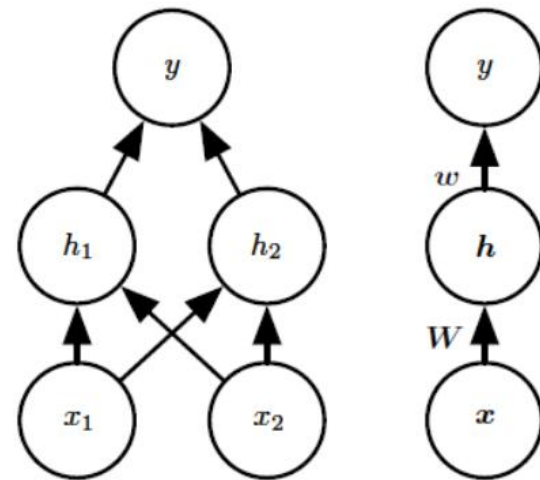
6.1 深度前馈网络：学习XOR函数

➤ 网络

$$f(x; W, c, w, b) = w^T \max\{0, W^T x + c\} + b$$

$$W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad c = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad w = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad b = 0$$

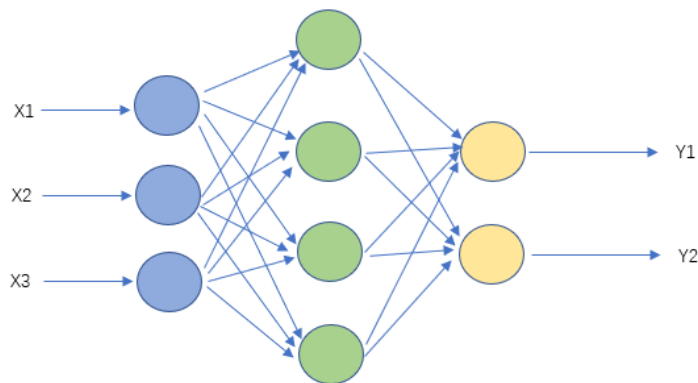
$$X = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \xrightarrow{\text{网络}} y = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$



6.1 深度前馈网络

➤ 深度前馈网络/前馈神经网络

- 目标是近似某个函数 f^*
- 称为**前馈**，模型的输出和模型本身之间没有反馈 (feedback) 连接
- 称为**网络**，因为用许多不同函数复合在一起来表示。链式结构是最常用的结构。 $f(x)=f^{(3)}(f^{(2)}(f^{(1)}(x)))$



如何让网络匹配 f^* ?

训练数据: 每个样本 x 带有标签

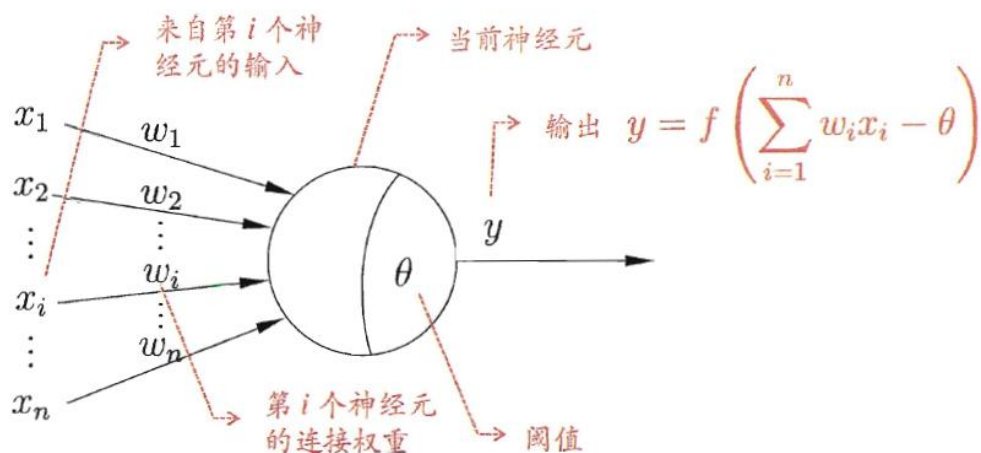
$$y=f(x) \approx f^*(x)$$

学习算法: 决定每层如何逼近 f^*

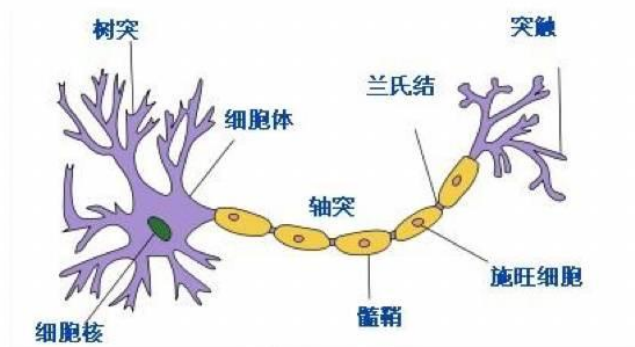
6.1 深度前馈网络

➤ 深度前馈网络/前馈神经网络

- 称为神经网络，每层由多个并行操作的单元组成，每个单元类似一个神经元



神经元

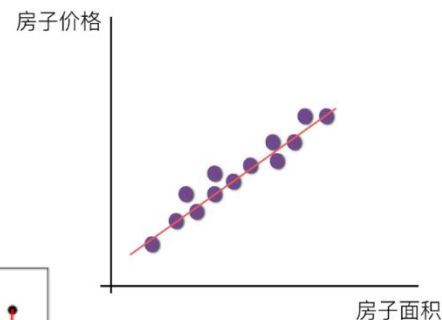
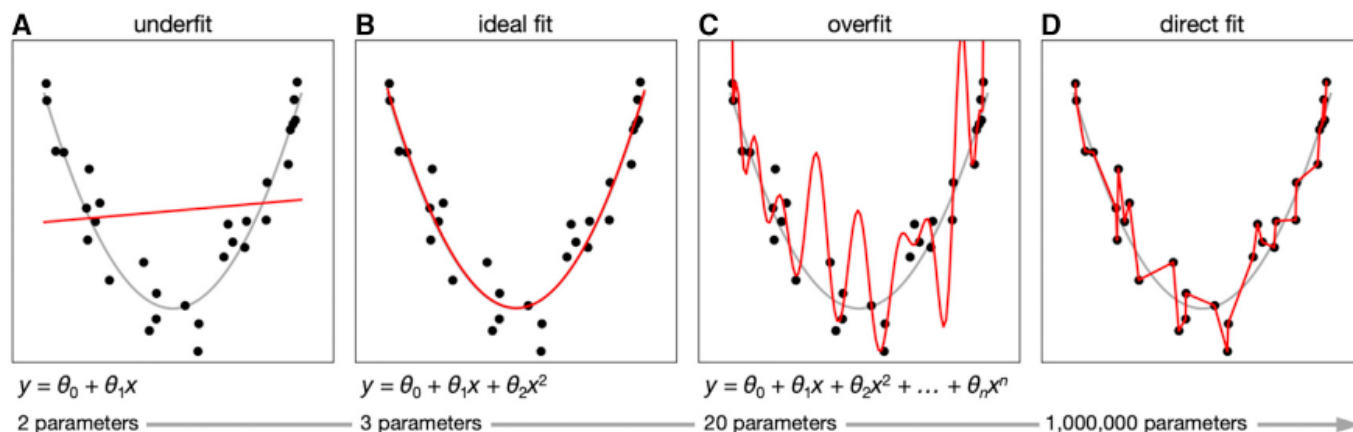


6.1 深度前馈网络

➤ 线性模型

- 线性回归，通过闭解形式或者凸优化来高效可靠地拟合
- 缺陷：模型能力局限在线性函数里，无法理解两个输入变量间的相互作用

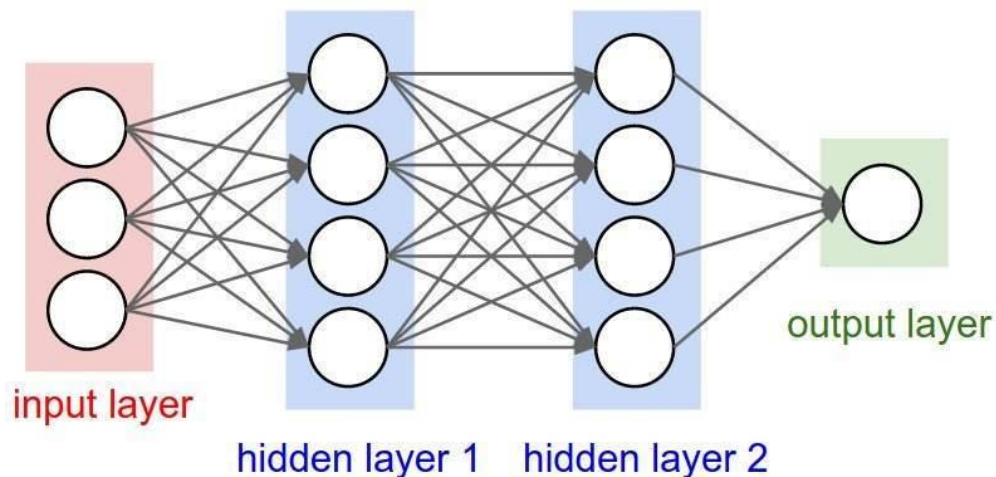
➤ 扩展到非线性函数



$\Phi(x)$: 非线性变换
如何选择 $\Phi(x)$?

6.2 深度前馈网络架构

- 输入单元
- 隐藏单元
- 输出单元
- 架构设计
- 神经网络训练
- 举例：手写数字识别



1) 隐藏单元

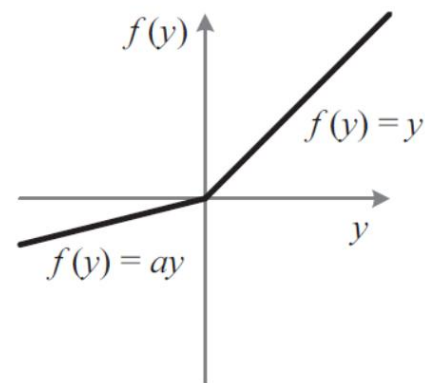
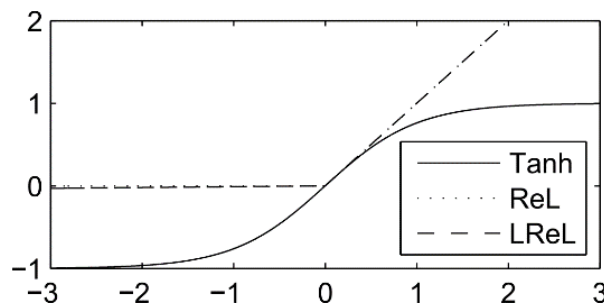
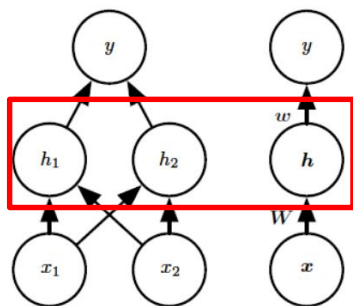
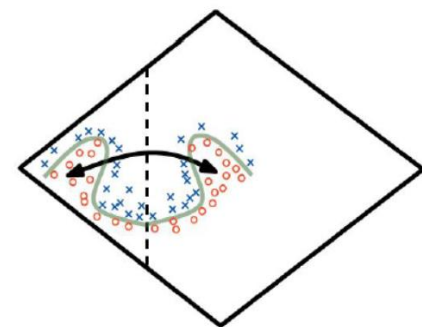
- 整流线性单元(ReLu)是激活函数极好的默认选择

$$g = \max\{0, z\}$$

- 扩展：当 $z_i < 0$ 时使用一个非零的斜率 a_i ：

$$h_i = g(z, a_i) = \max(0, z_i) + a_i \min(0, z_i)$$

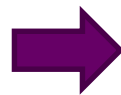
- 绝对值整流： $a_i = -1$
- 渗漏整流线性单元 Leaky ReLU: $a_i \approx 0.01$
- 参数化整流线性单元PReLU: a_i 作为学习的参数



1) 隐藏单元

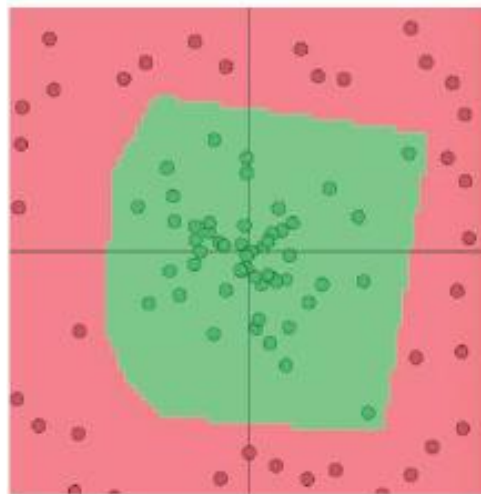
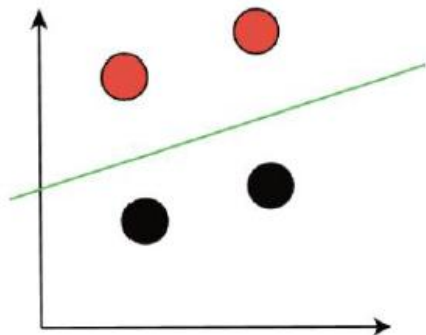
➤ 激活函数

Identity	Sigmoid	TanH	ArcTan
ReLU	Leaky ReLU	Randomized ReLU	Parameteric ReLU
Binary	Exponential Linear Unit	Soft Sign	Inverse Square Root Unit (ISRU)
Inverse Square Root Linear	Square Non-Linearity	Bipolar ReLU	Soft Plus



1) 隐藏单元

➤ 线性可分与不可分

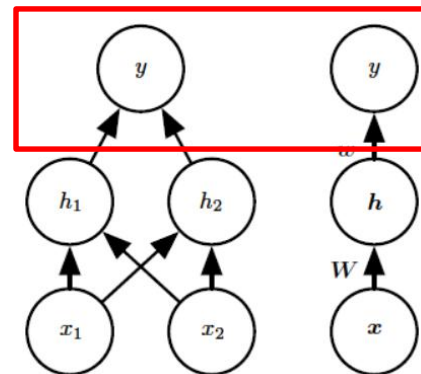
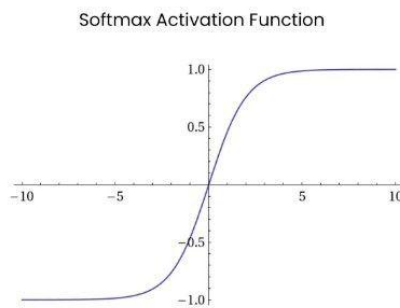
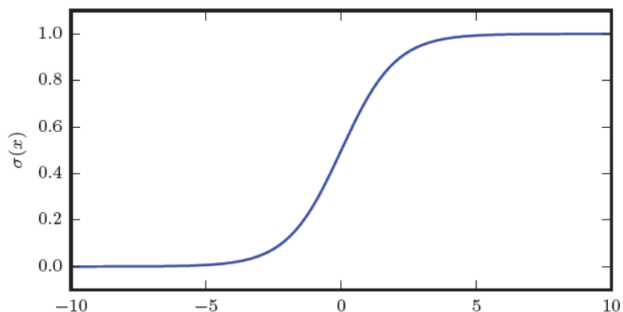


➤ 使用激活函数将输出变成输入的非线性函数，从而增强神经网络的表达能力

2) 输出单元

➤ 通过输出单元对隐藏特征进行一些额外的变换，来完成整个网络必须完成的任务

1. 用于高斯输出分布的线性单元: $\hat{y} = w^T h + b$
2. 用于Bernoulli输出分布的sigmoid单元(二分类问题; 抛硬币正反面): $\hat{y} = \sigma(w^T h + b)$ $\sigma(z) = \frac{1}{1 + \exp(-z)}$
3. 用于Multinoulli输出分布的softmax单元(多分类问题预测概率): $\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$



3) 架构设计

- 网络的整体结构：具有多少单元，以及单元应该如何连接
- 组织成层的单元组，层布置为链式结构，每一层是前一层的函数

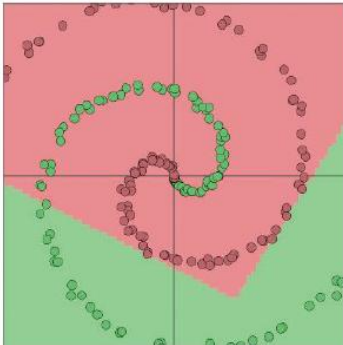
$$\begin{aligned}h^{(1)} &= g^{(1)}(W^{(1)T}x + b^{(1)}) \\h^{(2)} &= g^{(2)}(W^{(2)T}h^{(1)} + b^{(2)}) \\&\dots\dots\end{aligned}$$

- 选择网络深度和每层的宽度
- 单层网络足以表示任何函数，但网络层大到难以正确学习和泛化，使用更深的模型能减少单元数量，减少泛化误差

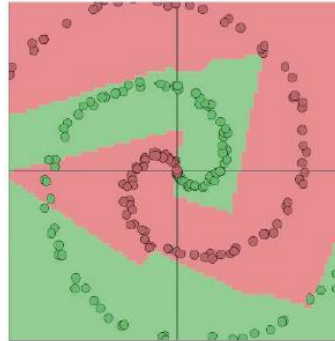
3) 架构设计

➤ 隐藏层神经元个数

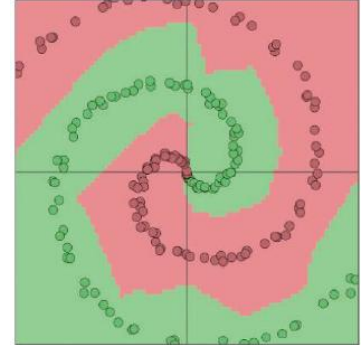
3个



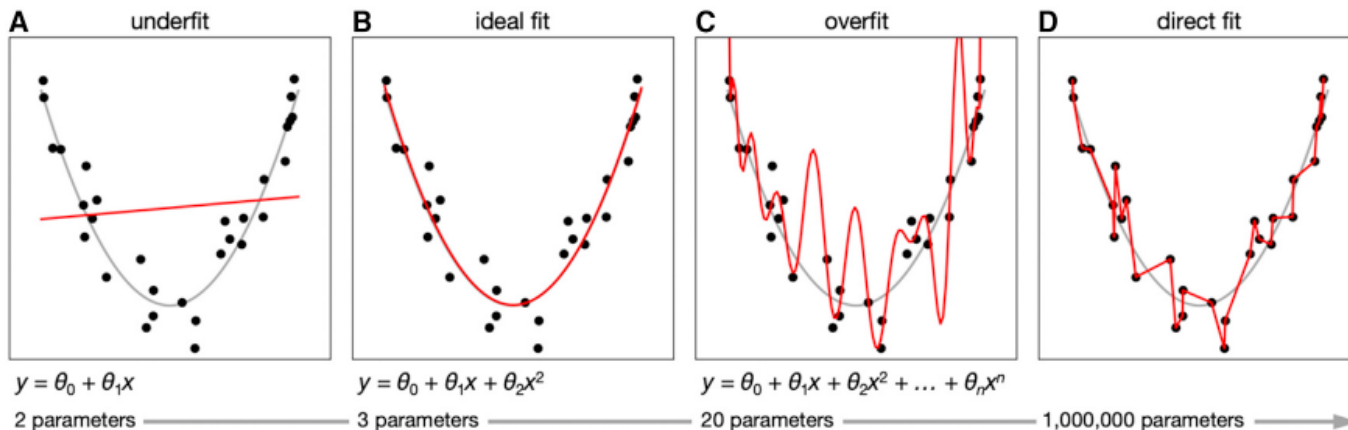
6个



20个

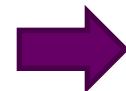


➤ 注意：防止过拟合！



4) 神经网络训练

- 损失函数：计算网络输出与目标之间的误差



- Softmax交叉熵
$$L = \frac{1}{N} \sum_{i=1}^N (-\log \frac{e^{l_{i,y_i}}}{\sum_{k=1}^C e^{l_{i,k}}})$$

- 均方误差
$$L = \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

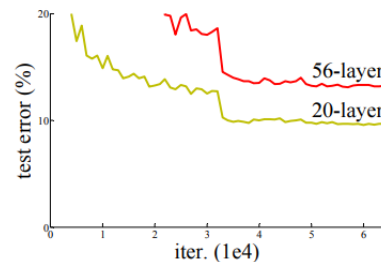
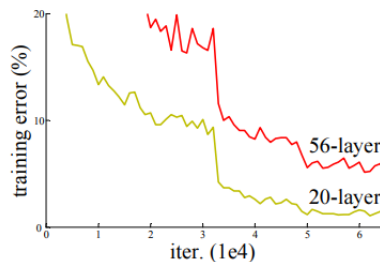
- 训练集： $\{\mathbf{x}_1, y_1\} \{\mathbf{x}_2, y_2\}, \dots$

- 优化器：修改神经网络的权值和偏置的方法和过程

- 将训练集输入网络，根据网络输出与目标相比较，然后采用学习规则调整网络权值和偏置值

- Backpropagation算法：误差反向传播

- 验证集：避免网络过拟合



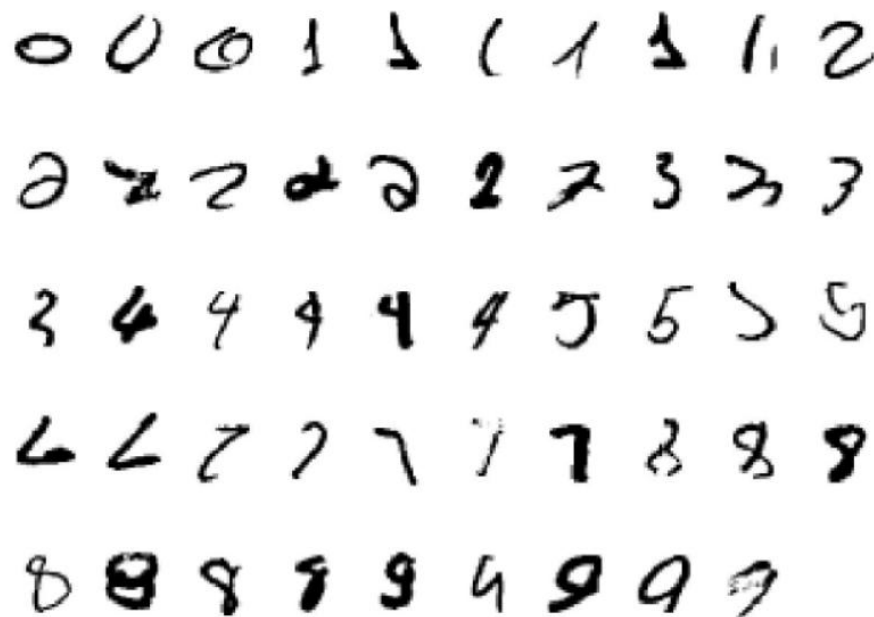
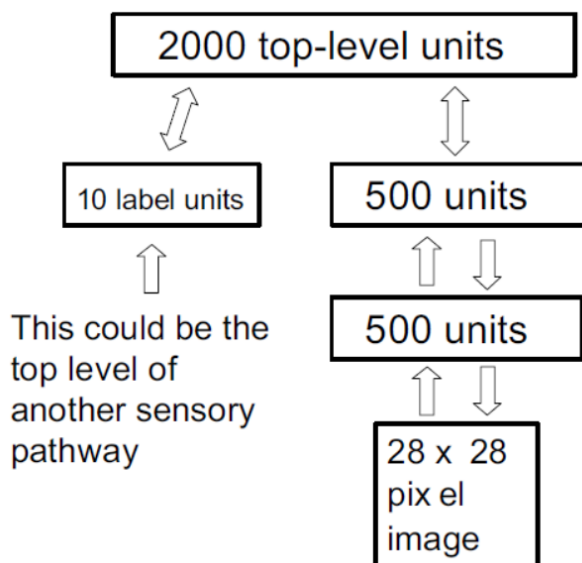
4) 神经网络训练

- 批量算法和小批量算法（优化器）
 - 批量或确定性梯度算法：使用整个训练集
 - 随机或者在线算法：每次只使用单个样本训练
 - 小批量或**小批量随机方法**：每次采用一个以上样本但又不是全部
- 优化方法：随机梯度下降SGD、动量方法Momentum等
- 批量大小：batch size（训练过程中1次迭代采用的样本数量）
- 周期：epoch（1个epoch有多次迭代使得整个训练集都被采用）

举例：手写数字识别

手写数字识别：10分类

- 输入 28×28 二值图像
- 数据集：60000张图
- 训练集：50000
- 验证集：10000



Geoffrey E. Hinton, "To Recognize Shapes, First Learn to Generate Images," 2006

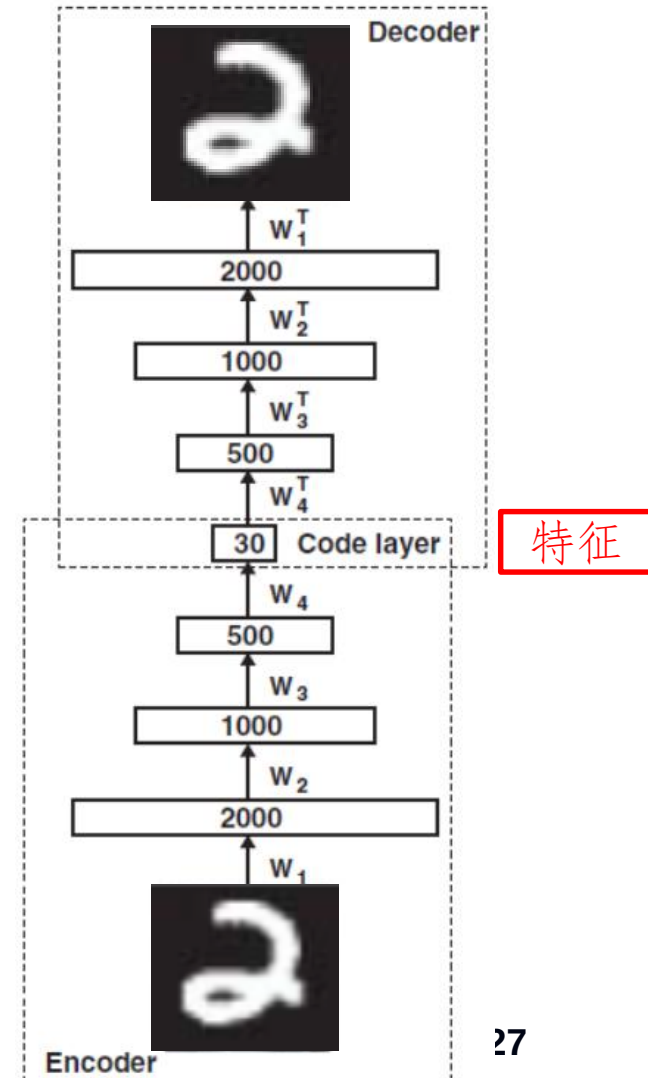
举例：手写数字识别

数据降维重构

- 手写数字
- 784-1000-500-250-30-

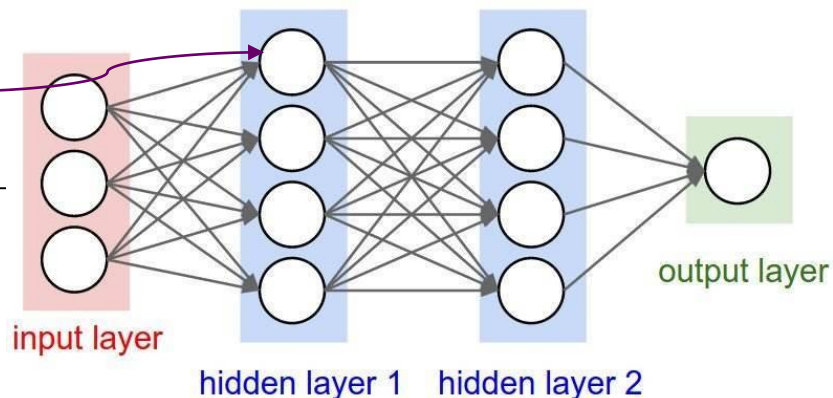


G. E. Hinton and R. R. Salakhutdinov, “Reducing the Dimensionality of Data with Neural Networks,” Science, 2006



总结

$$h = g(W^T x + b)$$



➤ 神经网络（全连接）架构

- 输入层
- 隐藏层：激活函数 (Relu)
- 输出层：线性单元、sigmoid单元、softmax单元

➤ 神经网络训练

- 优化：修改神经网络的权值(W)和偏置(b)的方法和过程
- 损失函数：计算网络输出与目标之间的误差，交叉熵 loss, MSE loss
- batch size、epoch
- 欠拟合与过拟合、泛化性能

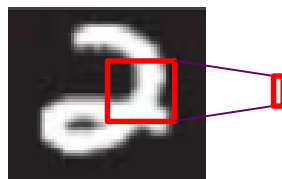
6.3 卷积神经网络

➤ 全连接网络缺点

- 图像很大，一个全连接层包含几万个连接权重，需要大数据集进行训练
- 对输入的变换和局部失真不具有不变性，要求输入大小归一化且位于中心
- 全连接层忽略输入的拓扑结构，如二维图像的局部空间相关性（邻近变量可以划分为小类别，如边缘、角）

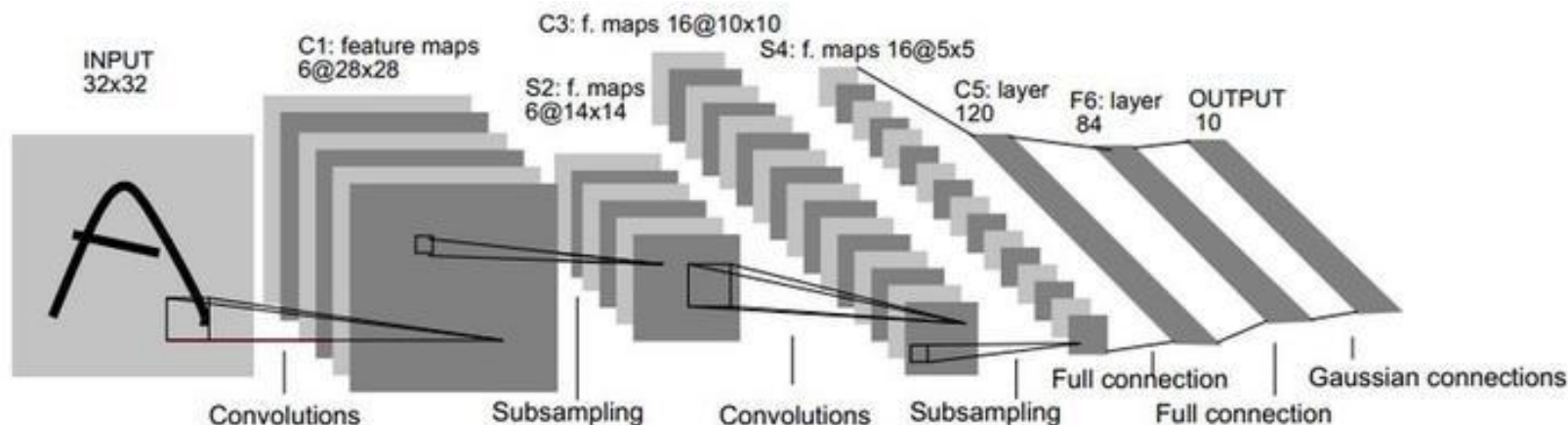
➤ 卷积网络

- 通过限制隐藏单元的**感受野到局部区域**，从而提取局部特征

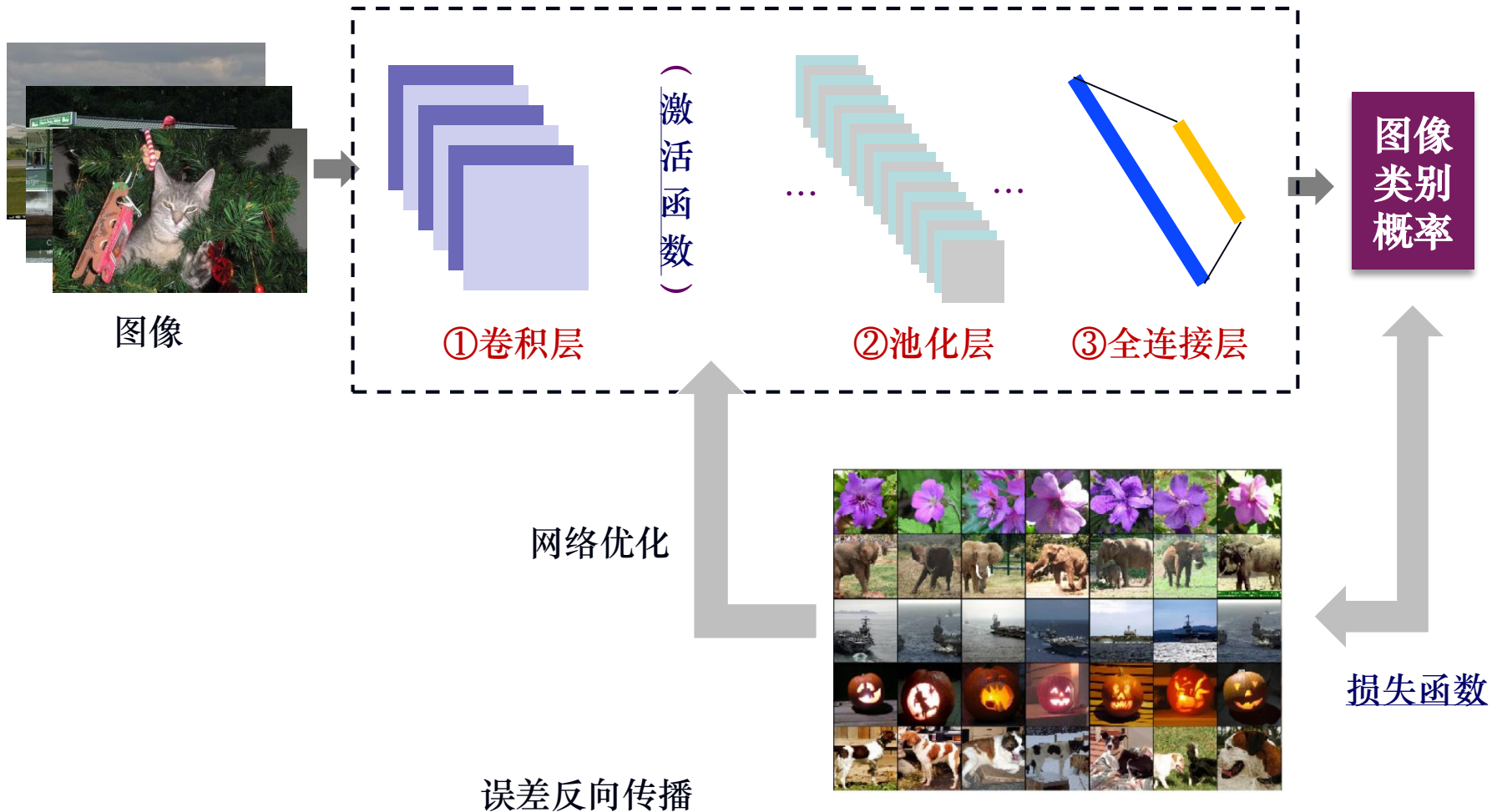


6.3 卷积神经网络

- 卷积神经网络(convolutional neural network, CNN)
 - 又称为卷积网络(convolutional network)，是一种专门用来处理具有类似网格结构数据的神经网络。例如，时间序列数据（可认为是在时间轴上有规律地采样形成的一维网格）和图像数据（可看作是二维的像素网格）
 - 卷积神经网络是指至少在网络的一层中使用卷积运算来替代一般的矩阵乘法运算的神经网络

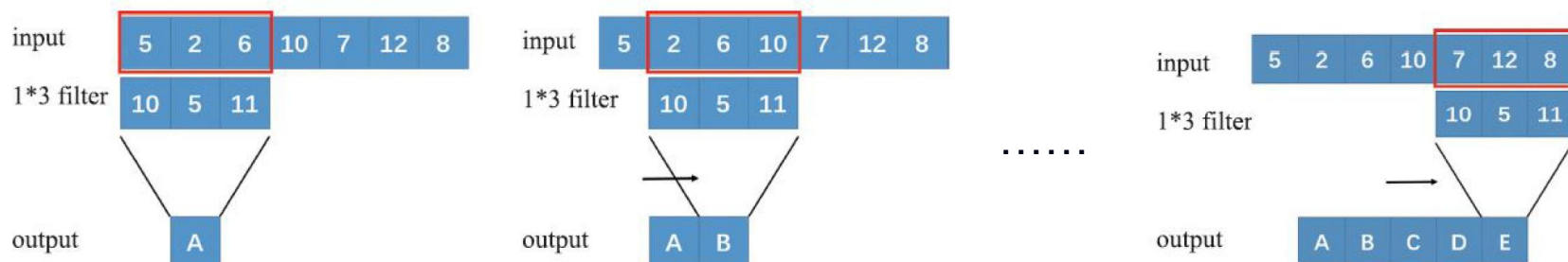


卷积神经网络的组成

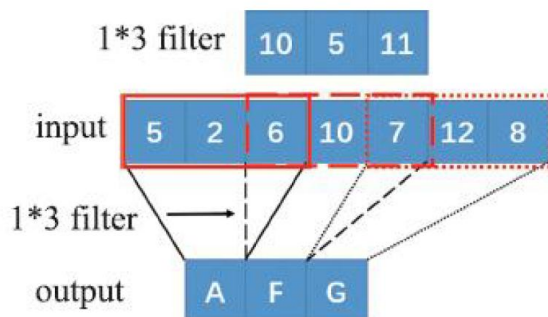


1) 卷积层

- 卷积层(Convolution): 对原图像进行卷积操作, 保留输入图像的空间特点
- 感受野: 卷积核每次滑动覆盖的格子范围

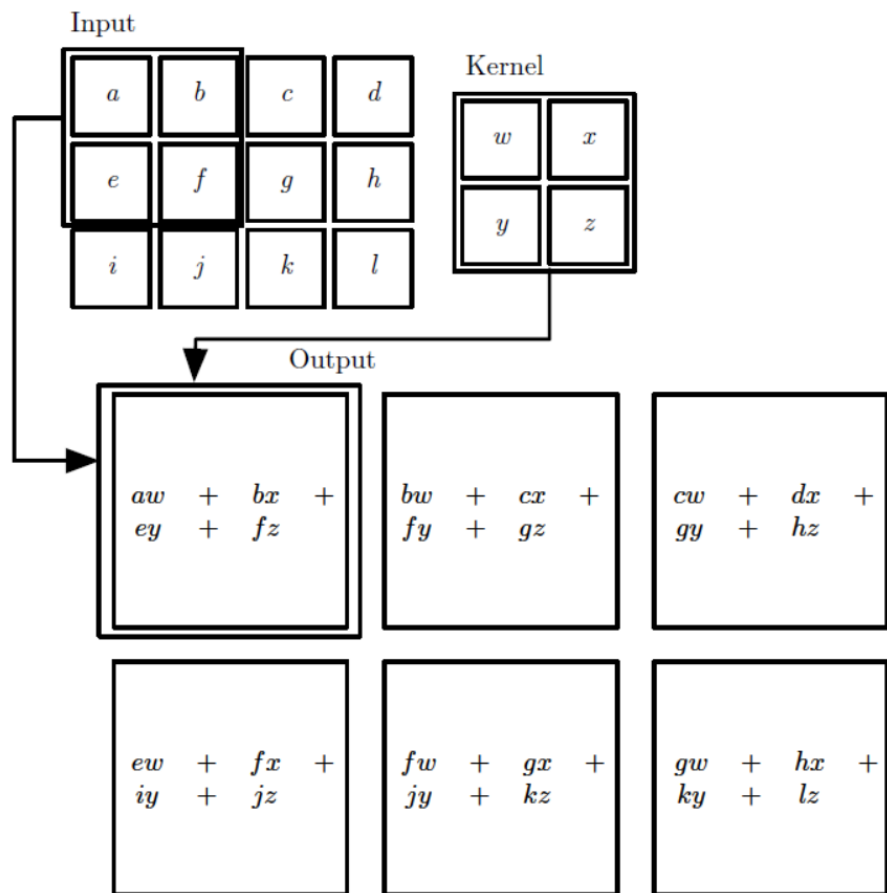


- 滑动步长



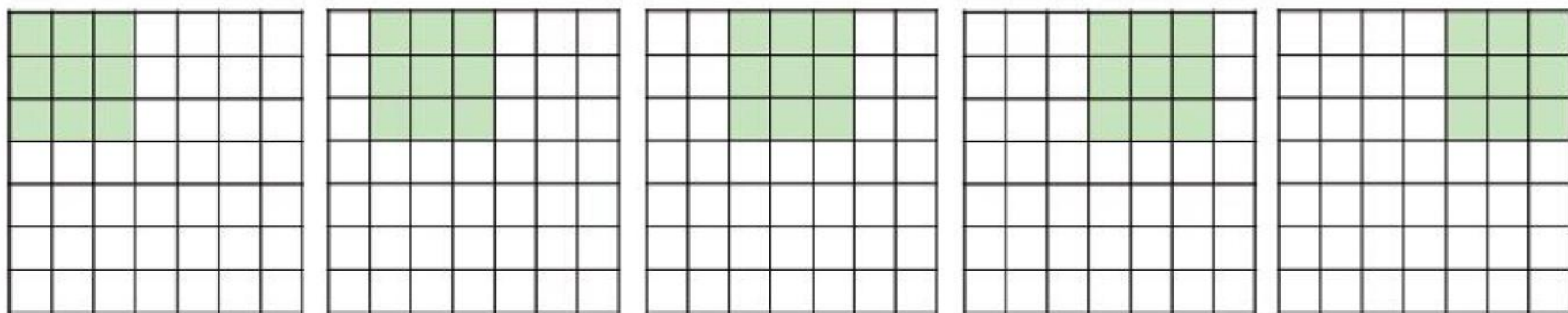
1) 卷积层

- 二维卷积: $\text{kernel}=2*2$, $\text{stride}=1$

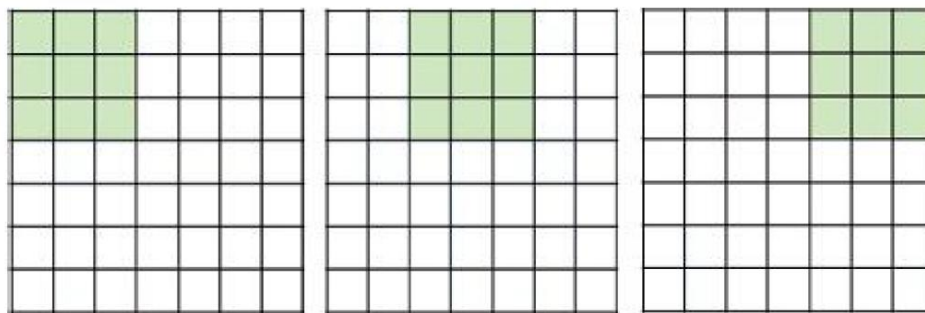


1) 卷积层

- 二维卷积: $\text{kernel}=3*3$, $\text{stride}=1$

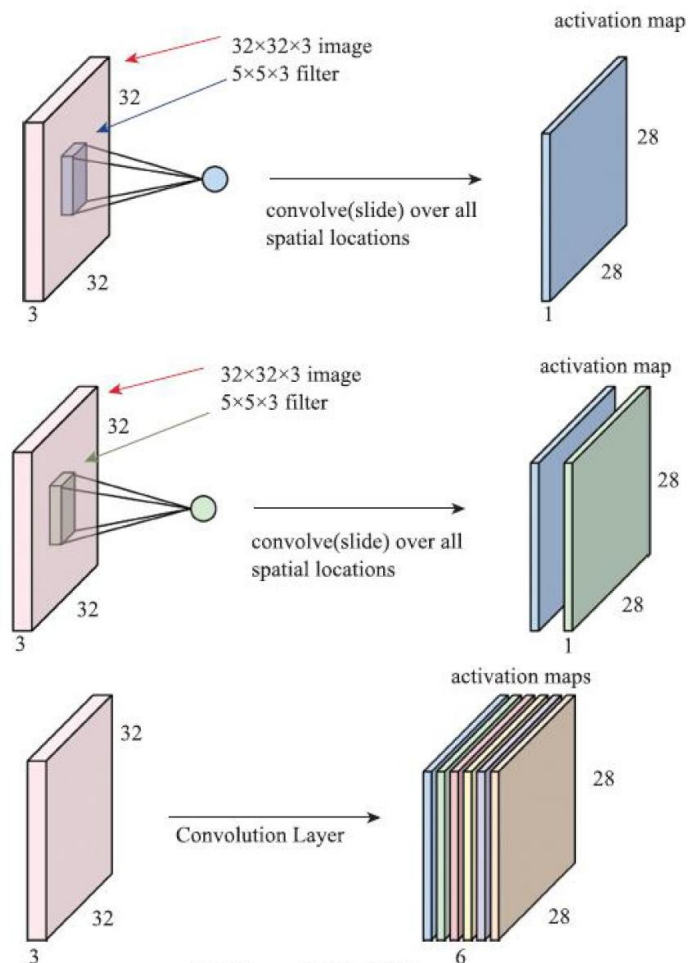
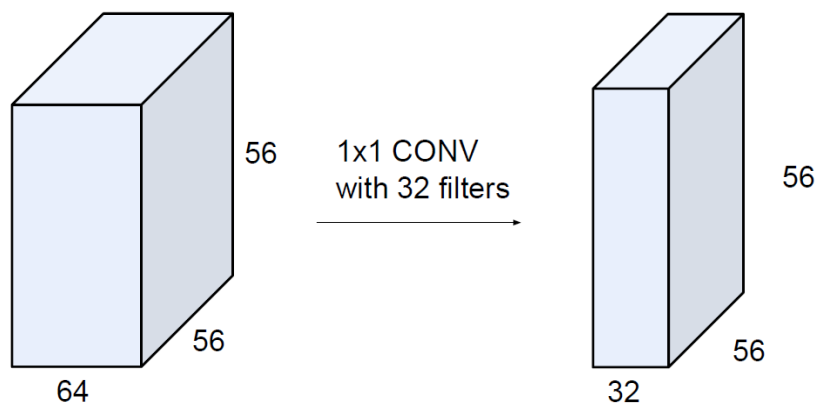


- 二维卷积: $\text{kernel}=3*3$, $\text{stride}=2$



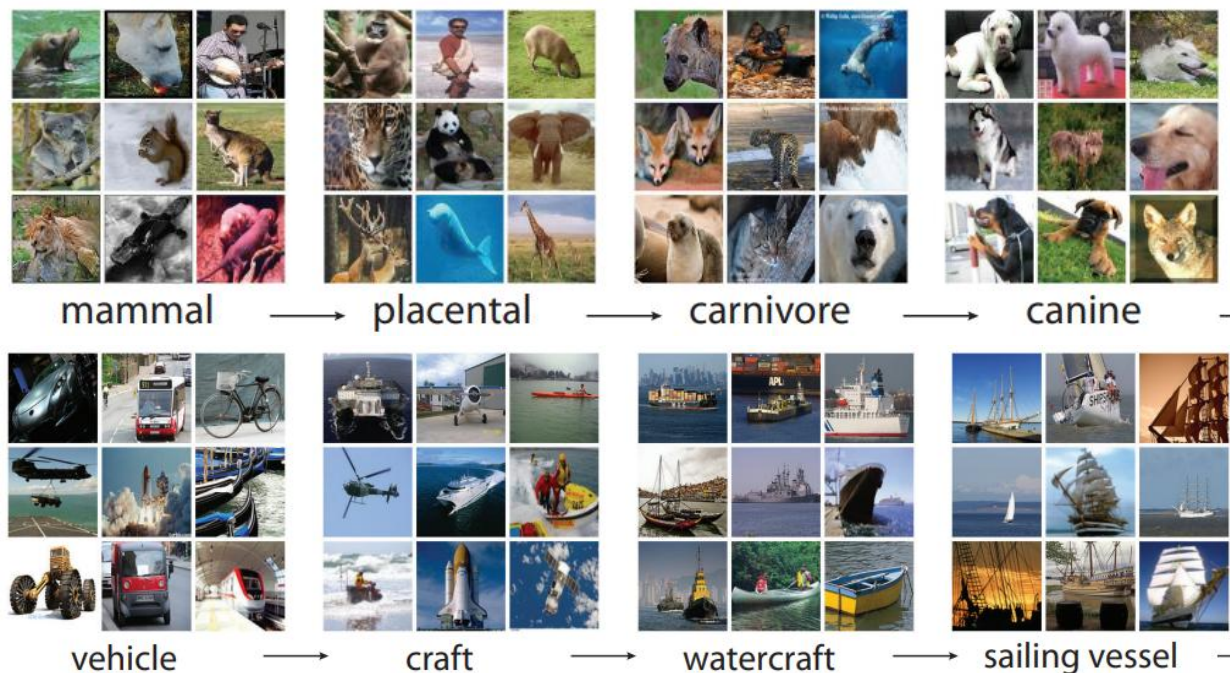
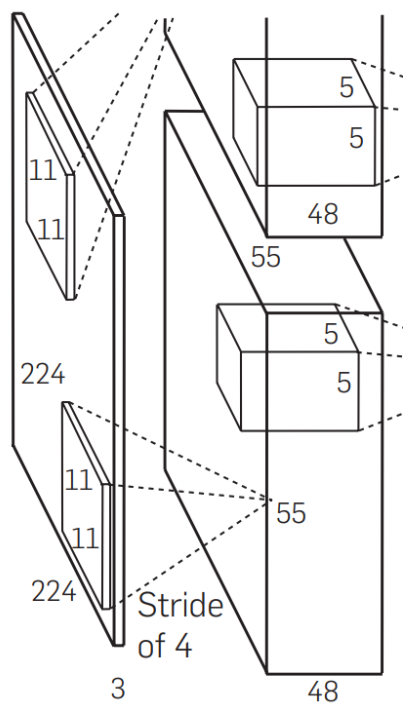
1) 卷积层

- 三维卷积: $\text{kernel}=5*5*3$, $\text{stride}=1$
- 三维卷积: $\text{kernel}=1*1*n$
 - 作用: 减小网络深度



1) 卷积层

- 卷积核举例：96个 $11 \times 11 \times 3$ 卷积核from AlexNet
- 输入： $224 \times 224 \times 3$ 图像



ImageNet Classification: 1000种类别

训练集: 1,281,167; 验证集: 50000; 测试集: 100000 张图像

1) 卷积层

- 卷积核举例：96个 $11 \times 11 \times 3$ 卷积核from AlexNet



ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky et al. 2012]

1) 卷积层

➤ 特征图(feature map)尺寸计算

- 输入图像尺寸: $W_1 \times H_1 \times D_1$
- 卷积核尺寸: $F \times F \times D_1$
- 卷积核个数: K
- 卷积核移动步长: S
- 填充pad大小: P

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$

$$D_2 = K$$

0	0	0	0	0	0			
0								
0								
0								
0								

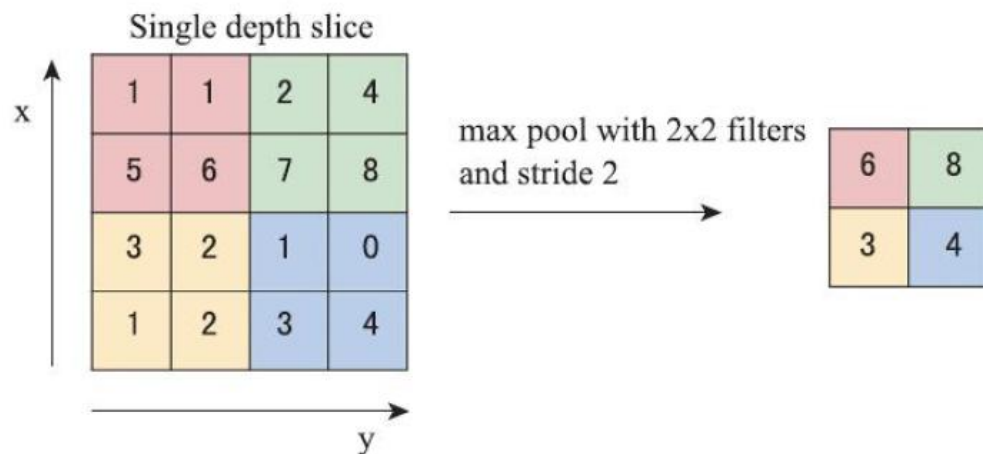
2) 池化层

- 池化（Pooling）函数使用某一位置的相邻输出的总体统计特征来代替网络在该位置的输出
- 例如，相邻矩形区域内的最大值(最大池化，max-pooling)、平均值(平均池化，mean-pooling)、L2 范数以及基于据中心像素距离的加权平均函数
- 输入图像尺寸： $W_1 \times H_1 \times D_1$
- 池化核尺寸： $F \times F$
- 池化核移动步长： S

$$W_2 = \frac{W_1 - F}{S} + 1$$

$$H_2 = \frac{H_1 - F}{S} + 1$$

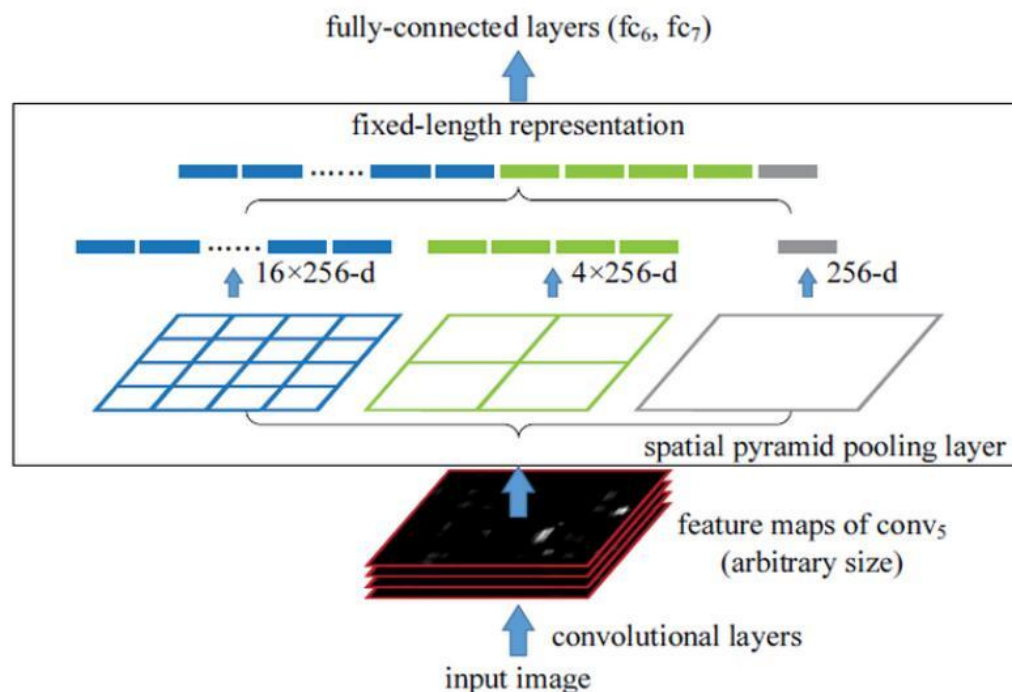
$$D_2 = D_1$$



2) 池化层

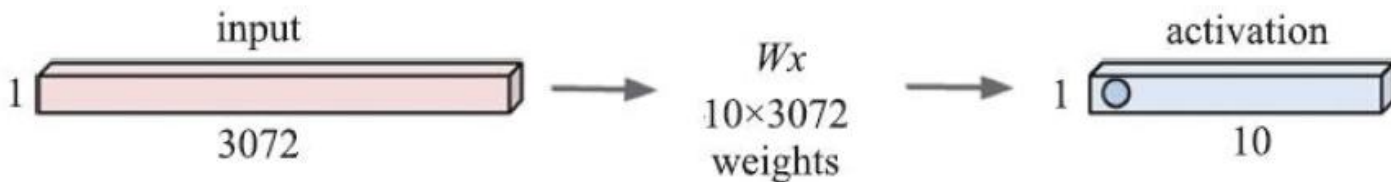
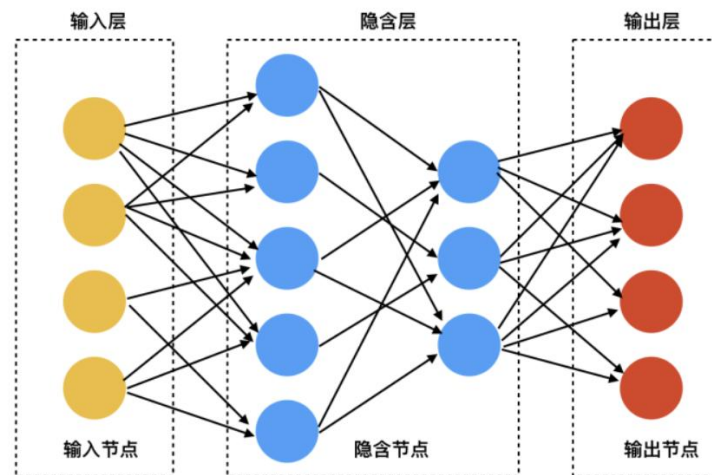
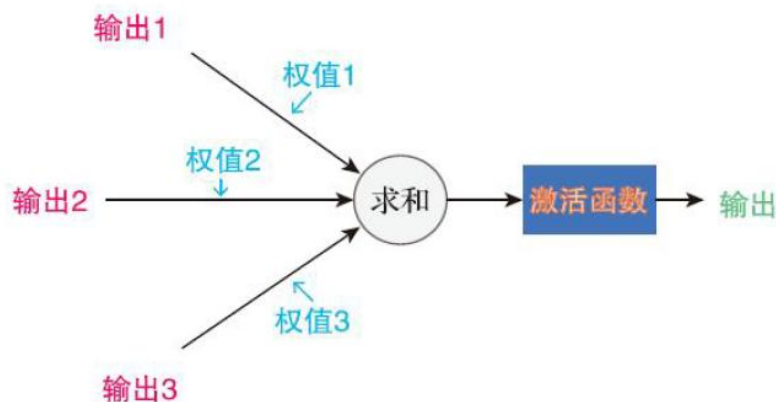
➤ 池化的作用

- 减小输入规模，提高统计效率，减少参数的存储需求
- 池化对于处理不同大小的输入具有重要作用



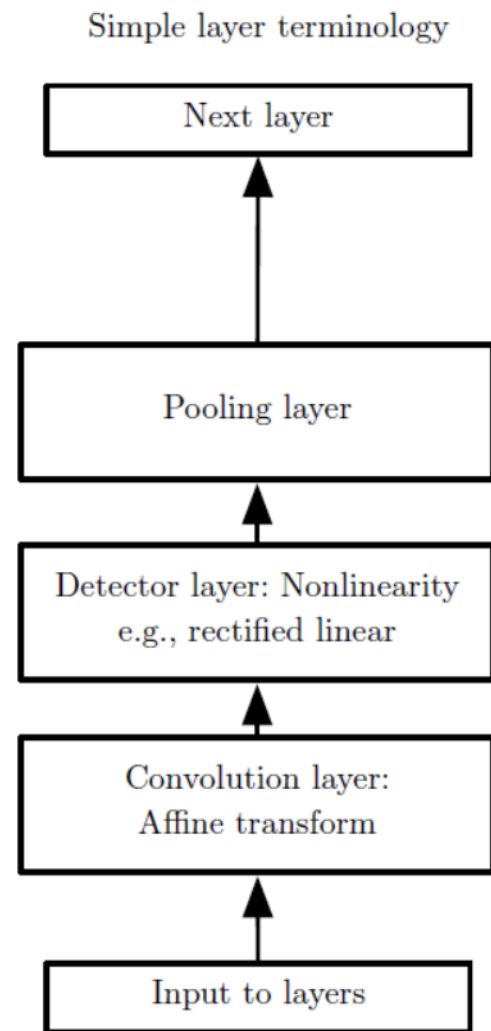
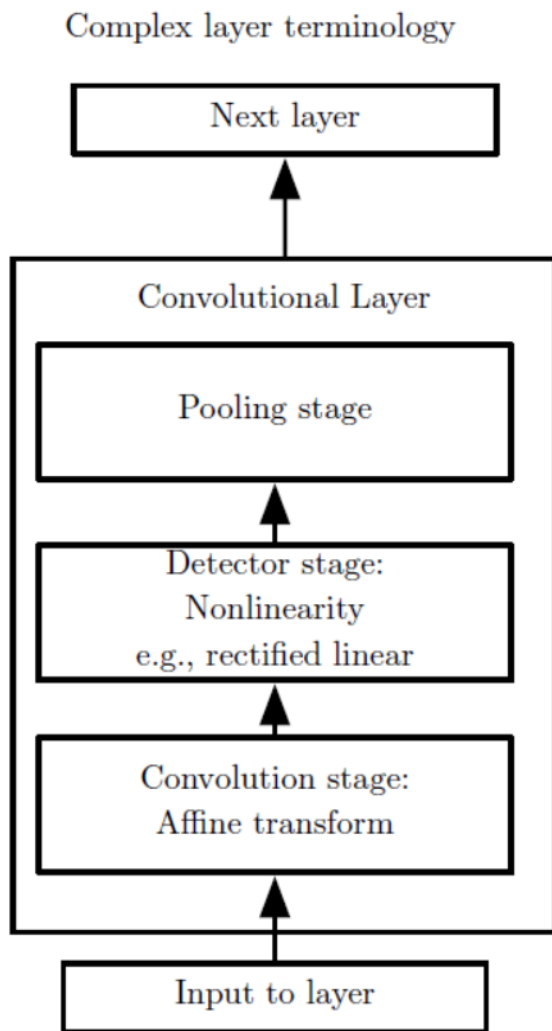
3) 全连接层

- 全连接层 (Fully Connected, FC): 类似于神经网络的隐含层, 将图像展开成1维向量作为输入
- 将该向量与权重向量做点乘, 经过激活得到结果

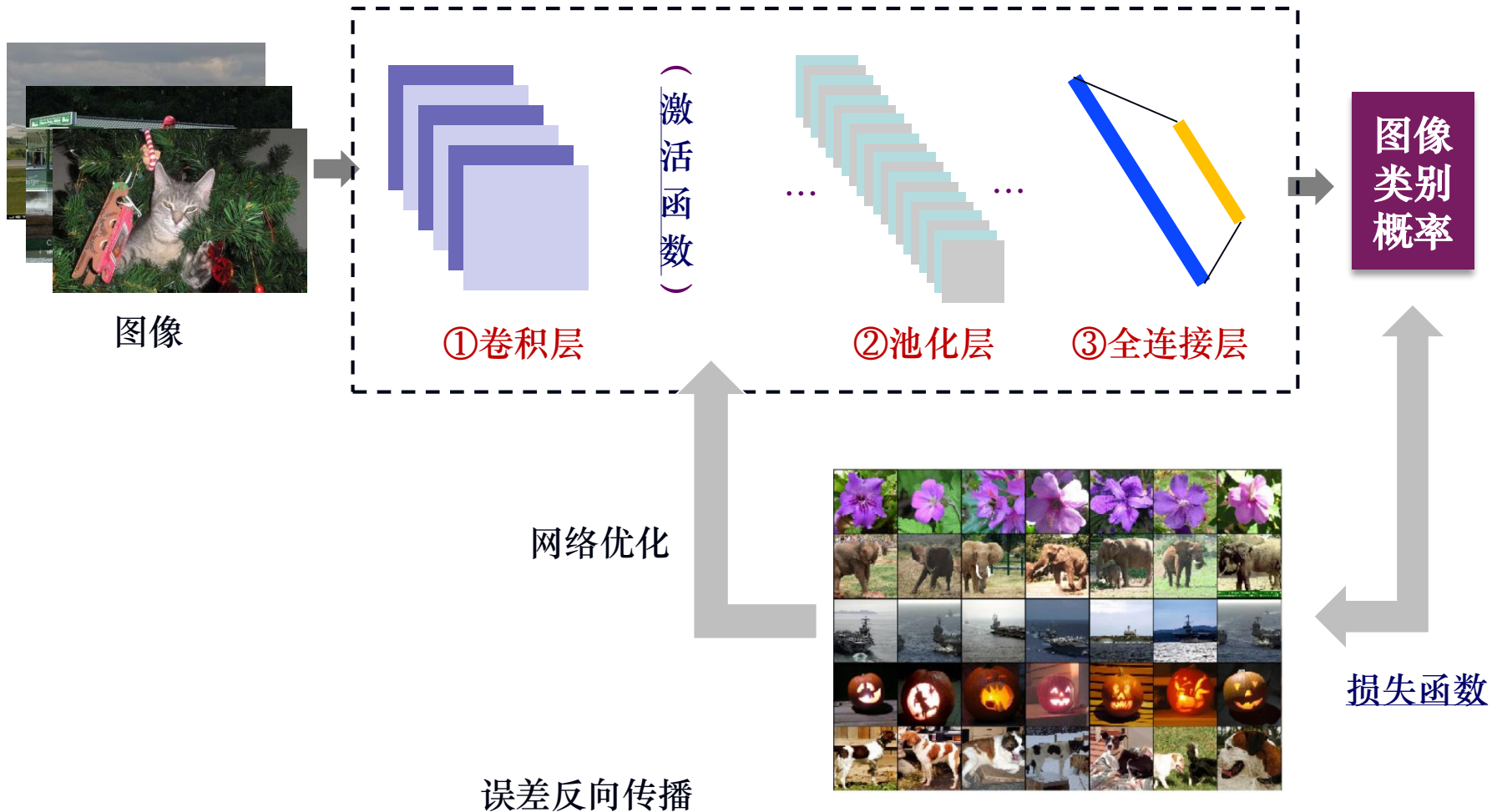


6.3 卷积神经网络

- 卷积网络中一个典型层包含三级
 1. 并行地计算多个卷积产生一组线性激活响应
 2. 每个线性激活响应将会通过一个**非线性的激活函数**，例如Relu函数，这一级有时也被称为探测级（detector stage）
 3. 使用池化函数来进一步调整这一层的输出
- 有时网络最后会有全连接层：类似于神经网络的隐藏层



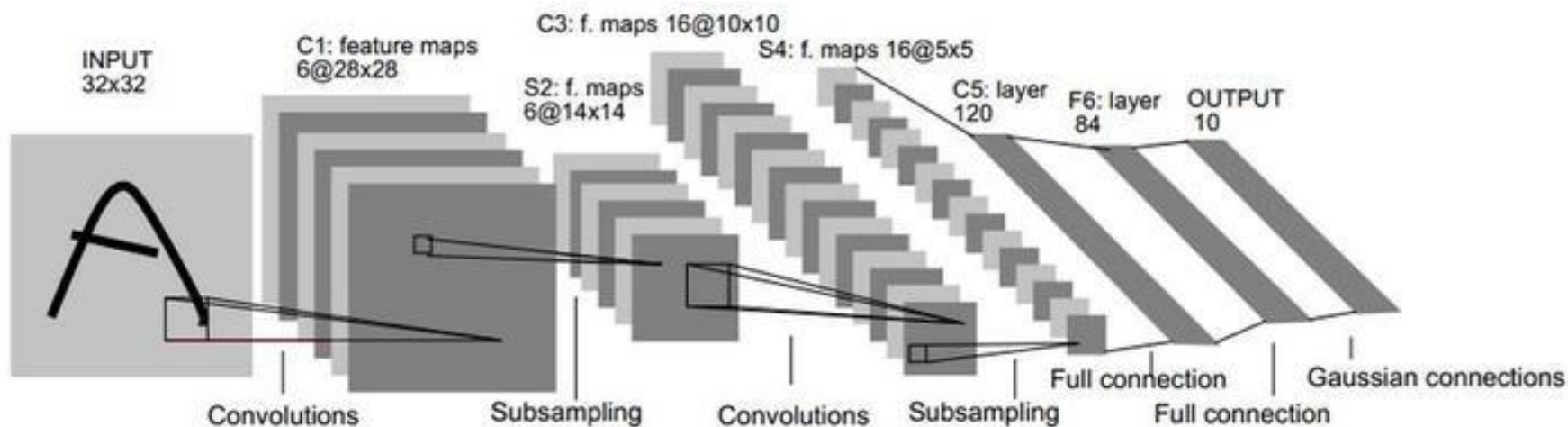
卷积神经网络的组成



举例：LeNet-5

- 网络输入：32*32的手写数字图像
- 网络输出：识别结果（分类结果，共10个）
- **C1+S2**：卷积核(5*5)*6，无padding，池化
- C3+S4：卷积核(5*5)*16，无padding，池化
- C5：卷积核(5*5*16)*120
- F6：84个节点对应7*12编码图

input: 32*32
 C1 output: 28*28*6
 训练参数：
 $(5*5+1)*6=156$
 S2 output: 14*14*6

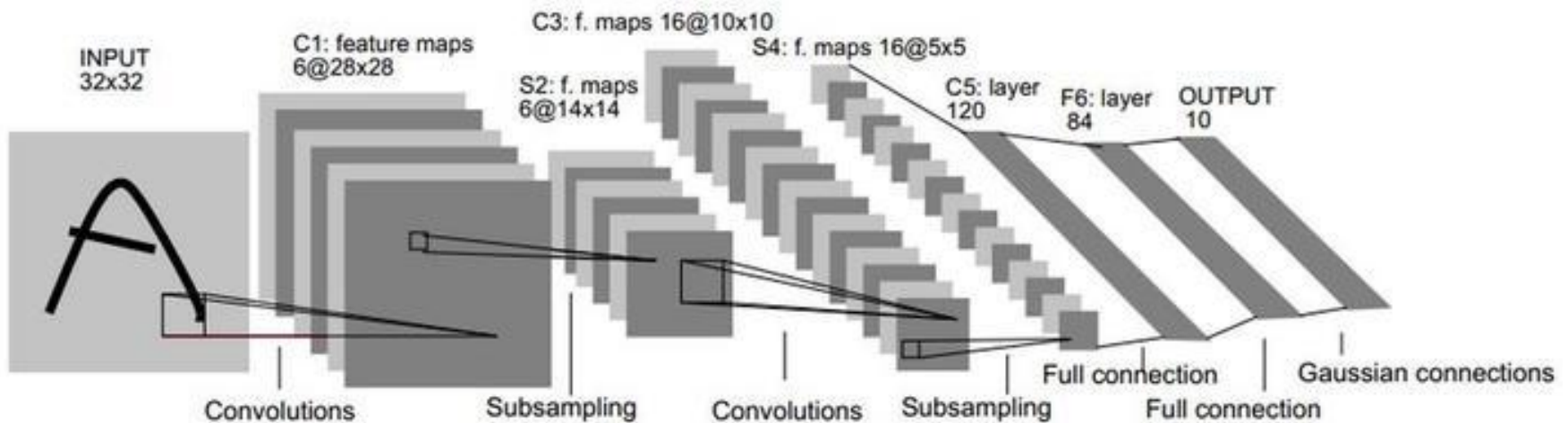


Gradient-based learning applied to document recognition [LeCun, Bottou, Bengio, Haffner 1998]

举例：LeNet-5

- 网络输入：32*32的手写数字图像
- 网络输出：识别结果（分类结果，共10个）
- C1+S2：卷积核(5*5)*6，无padding，池化
- **C3+S4**：卷积核(5*5)*16，无padding，池化
- C5：卷积核(5*5*16)*120
- F6：84个节点对应7*12编码图

S2 output: $14 \times 14 \times 6$
 C3 output: $10 \times 10 \times 16$
 训练参数：
 $(5 \times 5 \times 6 + 1) \times 16 = 2416$



Gradient-based learning applied to document recognition [LeCun, Bottou, Bengio, Haffner 1998]

举例：LeNet-5

- 网络输入：32*32的手写数字图像
- 网络输出：识别结果（分类结果，共10个）
- C1+S2：卷积核(5*5)*6，无padding，池化
- C3+S4：卷积核(5*5)*16，无padding，池化
- **C5**：卷积核(5*5*16)*120
- **F6**：84个节点对应7*12编码图

S4 output: $5*5*16$

C5 output: 120

训练参数:

$$(5*5*16+1)*120=48120$$

F6 output: 84

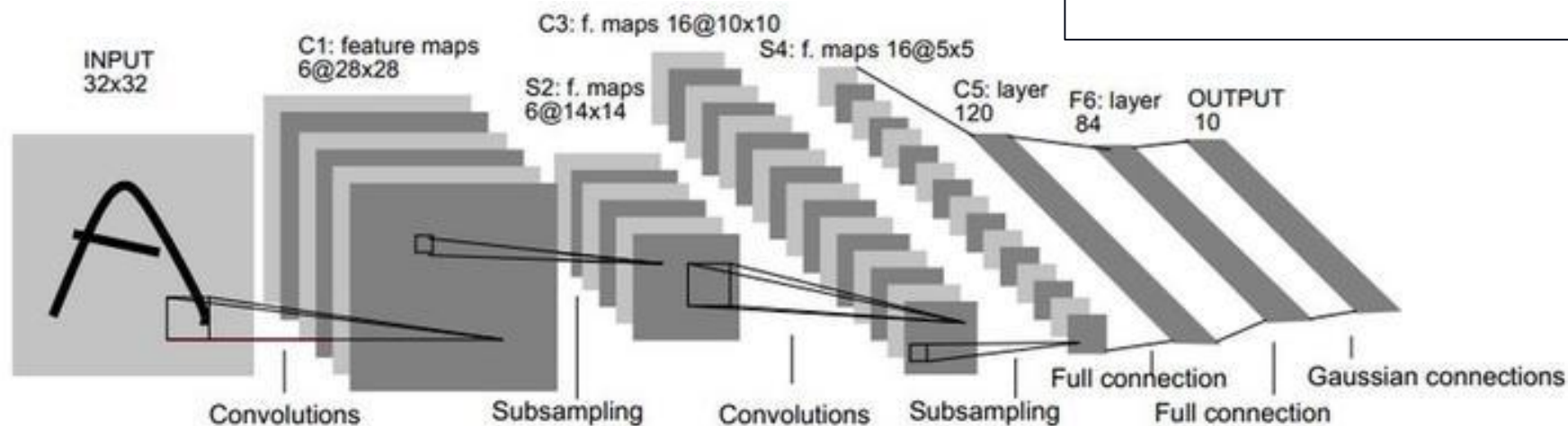
训练参数:

$$(120+1)*84=10164$$

OUTPUT: 10

训练参数:

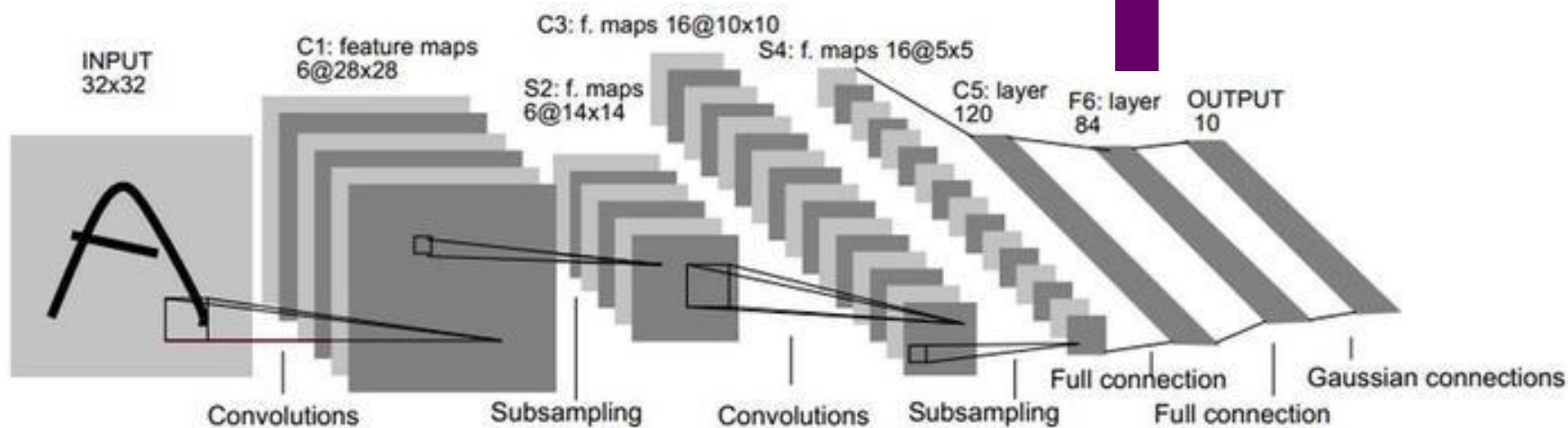
$$(84+1)*10=850$$



Gradient-based learning applied to document recognition [LeCun, Bottou, Bengio, Haffner 1998]

举例：LeNet-5

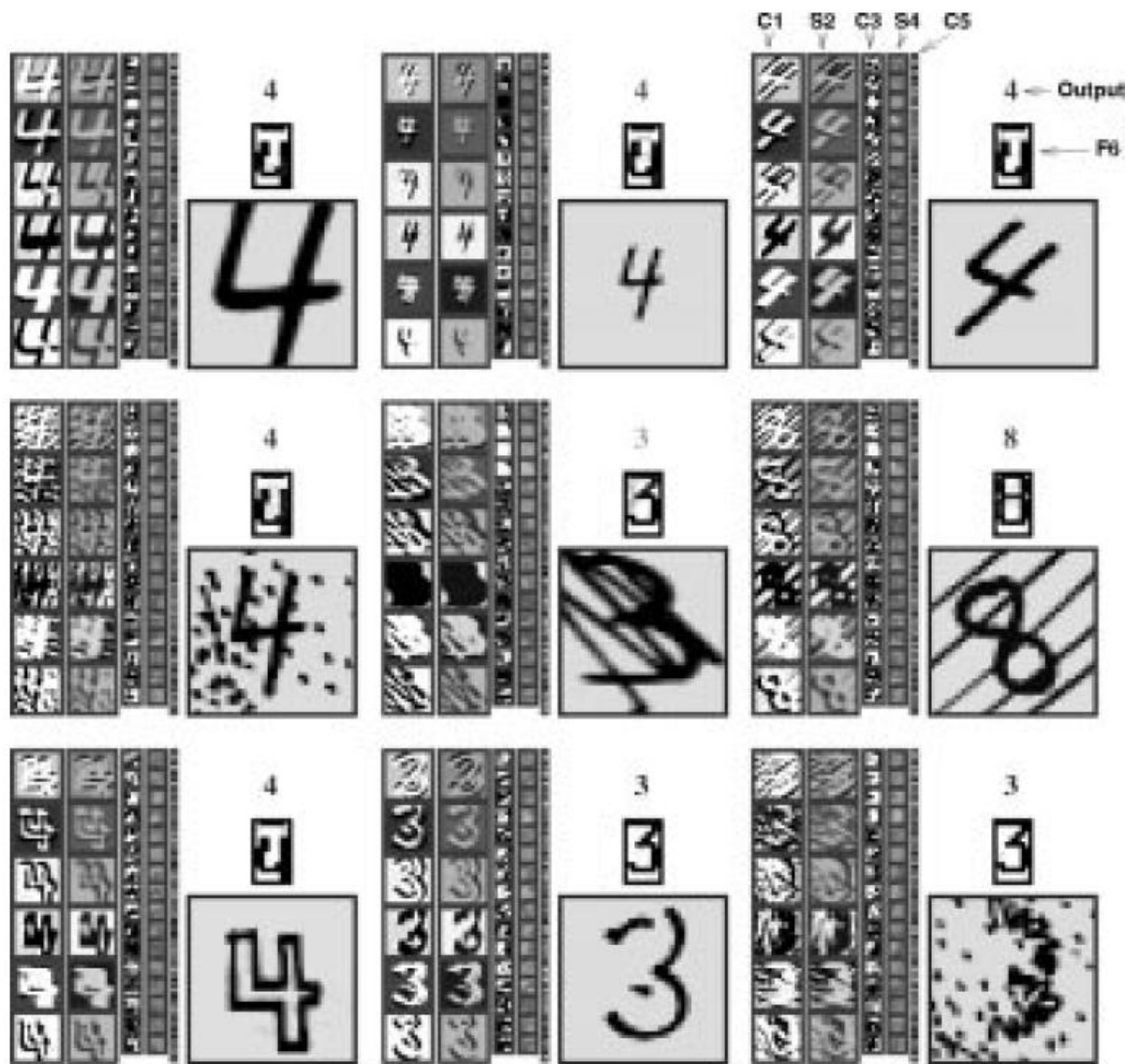
- 网络输入：32*32的手写数字图像
- 网络输出：识别结果
- C1+S2：卷积核(5*5)*6，池化
- C3+S4：卷积核(5*5)*16，池化
- C5：卷积核(5*5*16)*120
- F6：84个节点对应7*12编码图



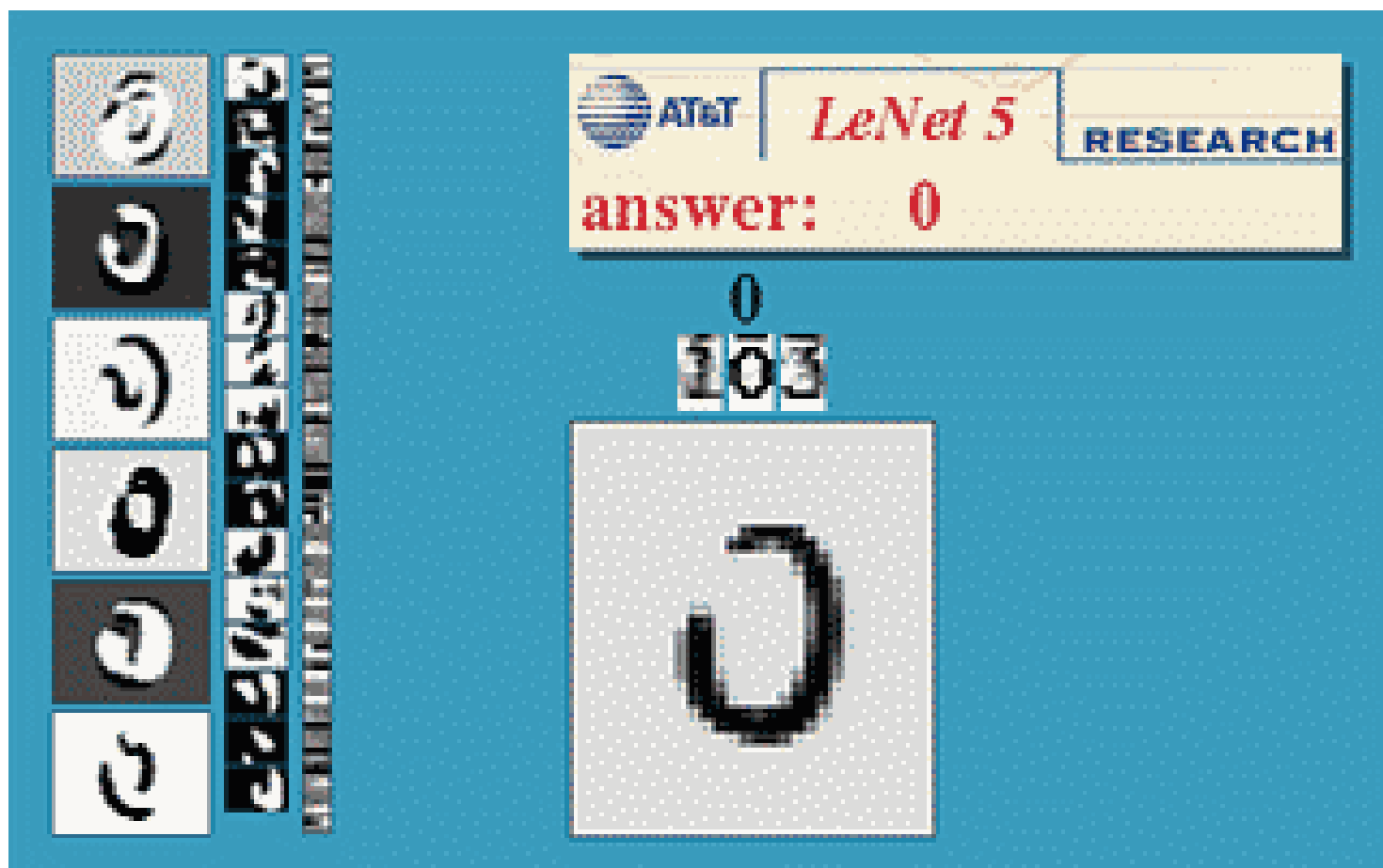
Gradient-based learning applied to document recognition [LeCun, Bottou, Bengio, Haffner 1998]

举例：LeNet

MNIST, 包含10个类别, 6万张 28×28 图像



举例：LeNet-5



6.4 深度学习框架

➤ PyTorch vs. TensorFlow vs. MindSpore

Caffe
(UC Berkeley)



Caffe2
(Facebook)
mostly features absorbed
by PyTorch



Torch
(NYU / Facebook)



PyTorch
(Facebook)

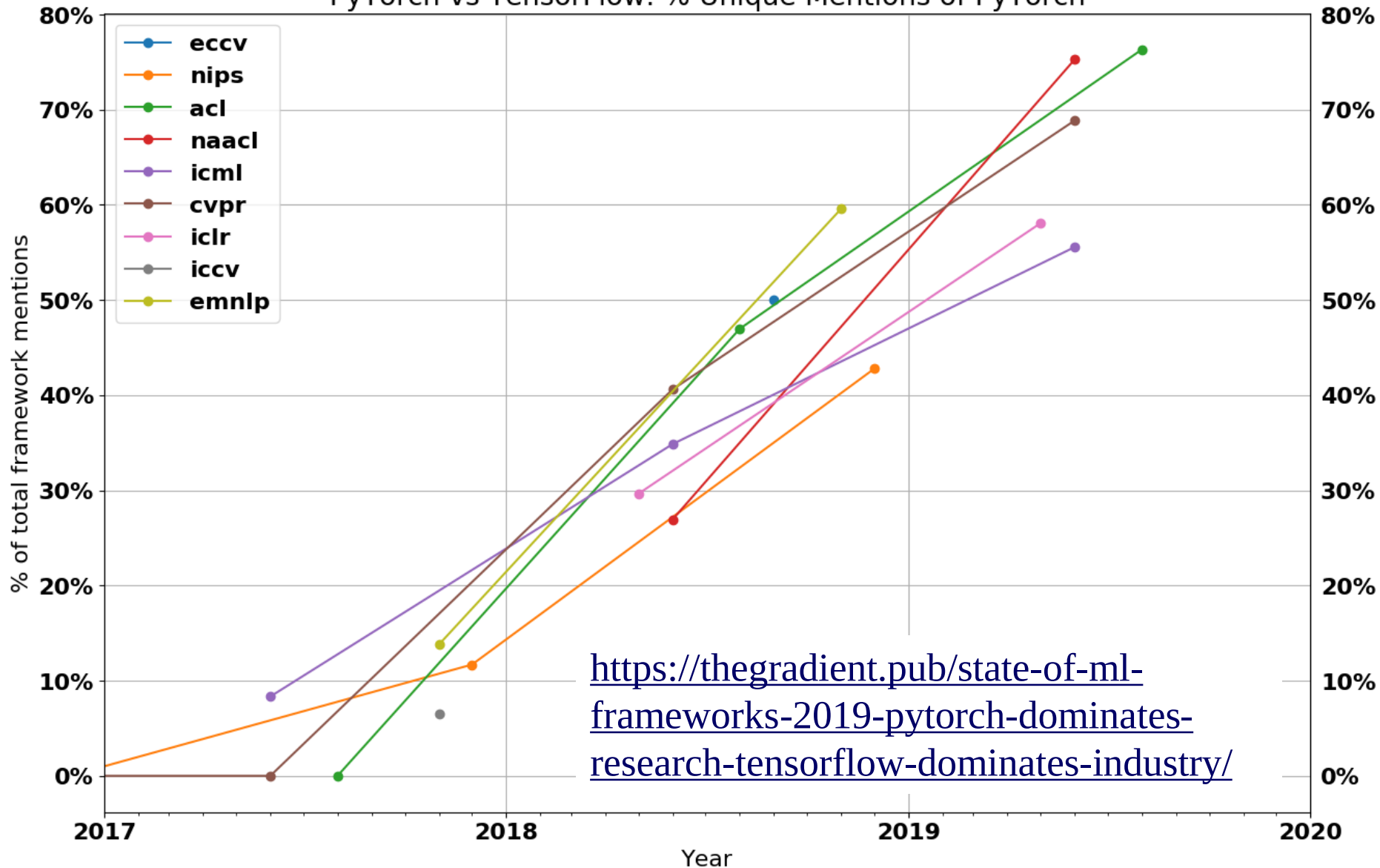
Theano
(U Montreal)



TensorFlow
(Google)



PyTorch vs TensorFlow: % Unique Mentions of PyTorch



6.4 深度学习框架

➤ PyTorch(PT) **vs.** TensorFlow(TF)

	PT	PT	PT	TF	TF	TF
CONFERENCE	2018	2019	GROWTH	2018	2019	GROWTH
CVPR	82	280	240%	116	125	7.7%
NAACL	12	66	450%	34	21	-38.2%
ACL	26	103	296%	34	33	-2.9%
ICLR	24	70	192%	54	53	-1.9%
ICML	23	69	200%	40	53	32.5%

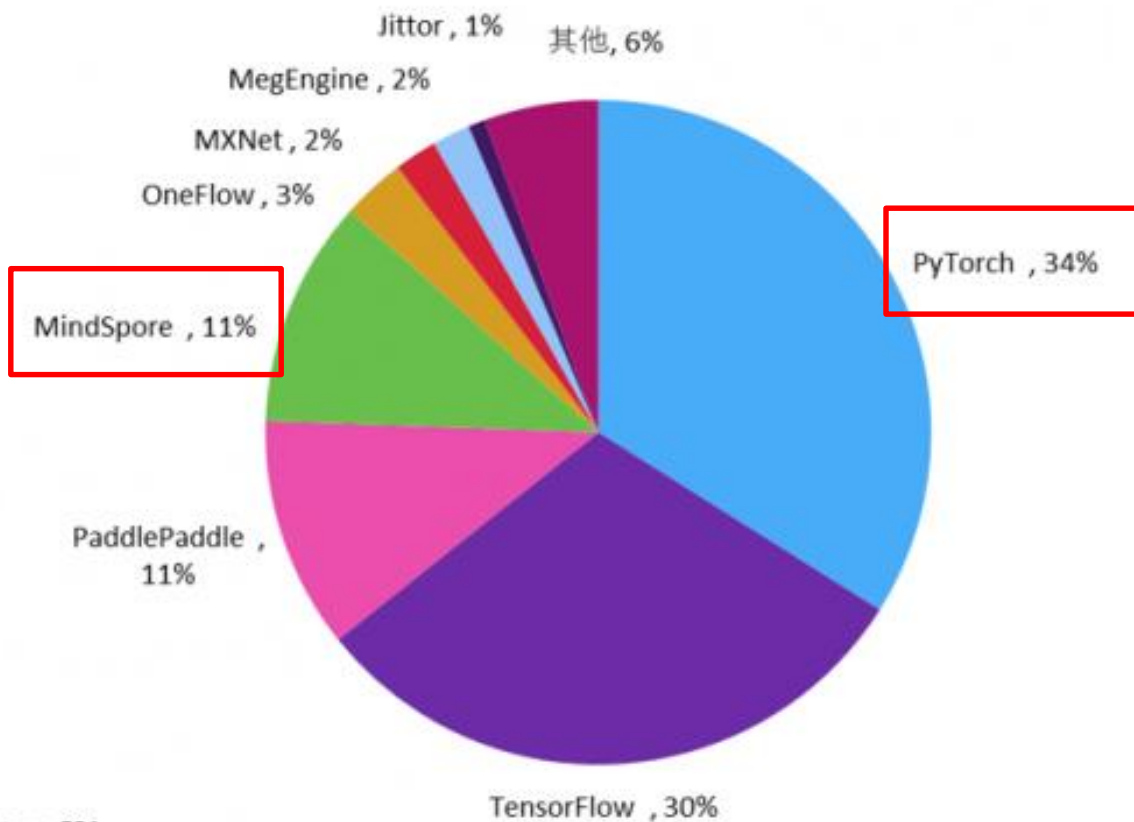
<https://thegradient.pub/state-of-ml-frameworks-2019-pytorch-dominates-research-tensorflow-dominates-industry/>

6.4 深度学习框架

➤ 行业研究机构 Omdia调研



Q11 中国开发者主流人工智能框架使用率排名

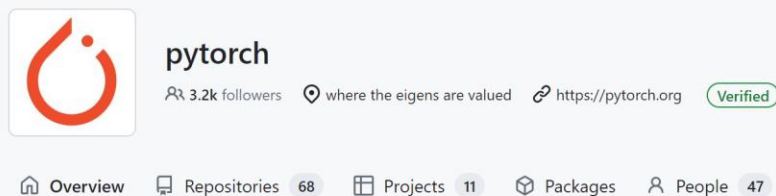


Note: n=531
Source: Omdia

© 2022 Omdia

PyTorch

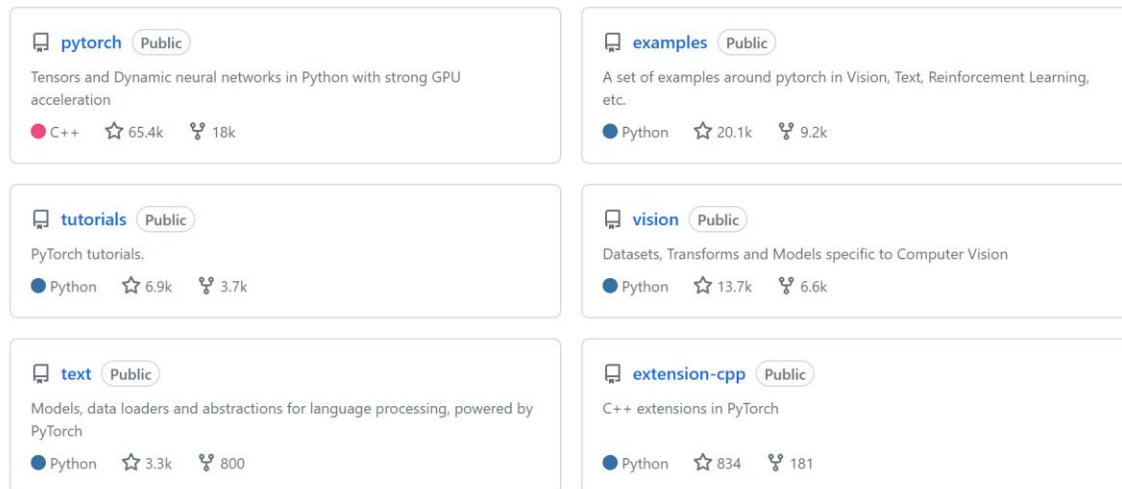
- <https://pytorch.org/>
- <https://github.com/pytorch/pytorch>



pytorch
3.2k followers · where the eigens are valued · <https://pytorch.org> Verified

Overview · Repositories 68 · Projects 11 · Packages · People 47

Pinned



pytorch Public Tensors and Dynamic neural networks in Python with strong GPU acceleration C++ 65.4k 18k	examples Public A set of examples around pytorch in Vision, Text, Reinforcement Learning, etc. Python 20.1k 9.2k
tutorials Public PyTorch tutorials. Python 6.9k 3.7k	vision Public Datasets, Transforms and Models specific to Computer Vision Python 13.7k 6.6k
text Public Models, data loaders and abstractions for language processing, powered by PyTorch Python 3.3k 800	extension-cpp Public C++ extensions in PyTorch Python 834 181

PyTorch Recipes [+]

Introduction to PyTorch [-]

Learn the Basics

Quickstart

Tensors

Datasets & DataLoaders

Transforms

Build the Neural Network

Automatic Differentiation with `torch.autograd`

Optimizing Model Parameters

Save and Load the Model

Introduction to PyTorch on YouTube [+]

Learning PyTorch [+]

Image and Video [-]

TorchVision Object Detection Finetuning Tutorial

Transfer Learning for Computer Vision Tutorial

Adversarial Example Generation

DCGAN Tutorial

Spatial Transformer Networks Tutorial

Optimizing Vision Transformer Model for Deployment



➤ <https://www.mindspore.cn/>

2.0.0-alpha ▾

"教程" 内搜索

欢迎查看MindSpore教程

初学教程 ▾

+ 基本介绍

[快速入门](#)

+ 张量 Tensor

+ 数据集 Dataset

+ 数据变换 Transforms

+ 网络构建

+ 函数式自动微分

+ 模型训练

+ 保存与加载

进阶教程 ▾

+ 高阶封装: Model

+ 模型模块自定义

+ 高级数据处理

+ 高级自动微分

+ 计算图

教程 (2.0.0-alpha) > 快速入门



在ModelArts平台运行



下载Notebook



下载样例代码



在Gitee上查看源文件

[基本介绍](#) || [快速入门](#) || [张量 Tensor](#) || [数据集 Dataset](#) || [数据变换 Transforms](#) || [网络构建](#) || [函数式自动微分](#) || [模型训练](#) || [保存与加载](#)

快速入门

本节通过MindSpore的API来快速实现一个简单的深度学习模型。若想要深入了解MindSpore的使用方法，请参阅各节最后提供的参考链接。

```
import mindspore
from mindspore import nn
from mindspore.dataset import vision, transforms
from mindspore.dataset import MnistDataset
```

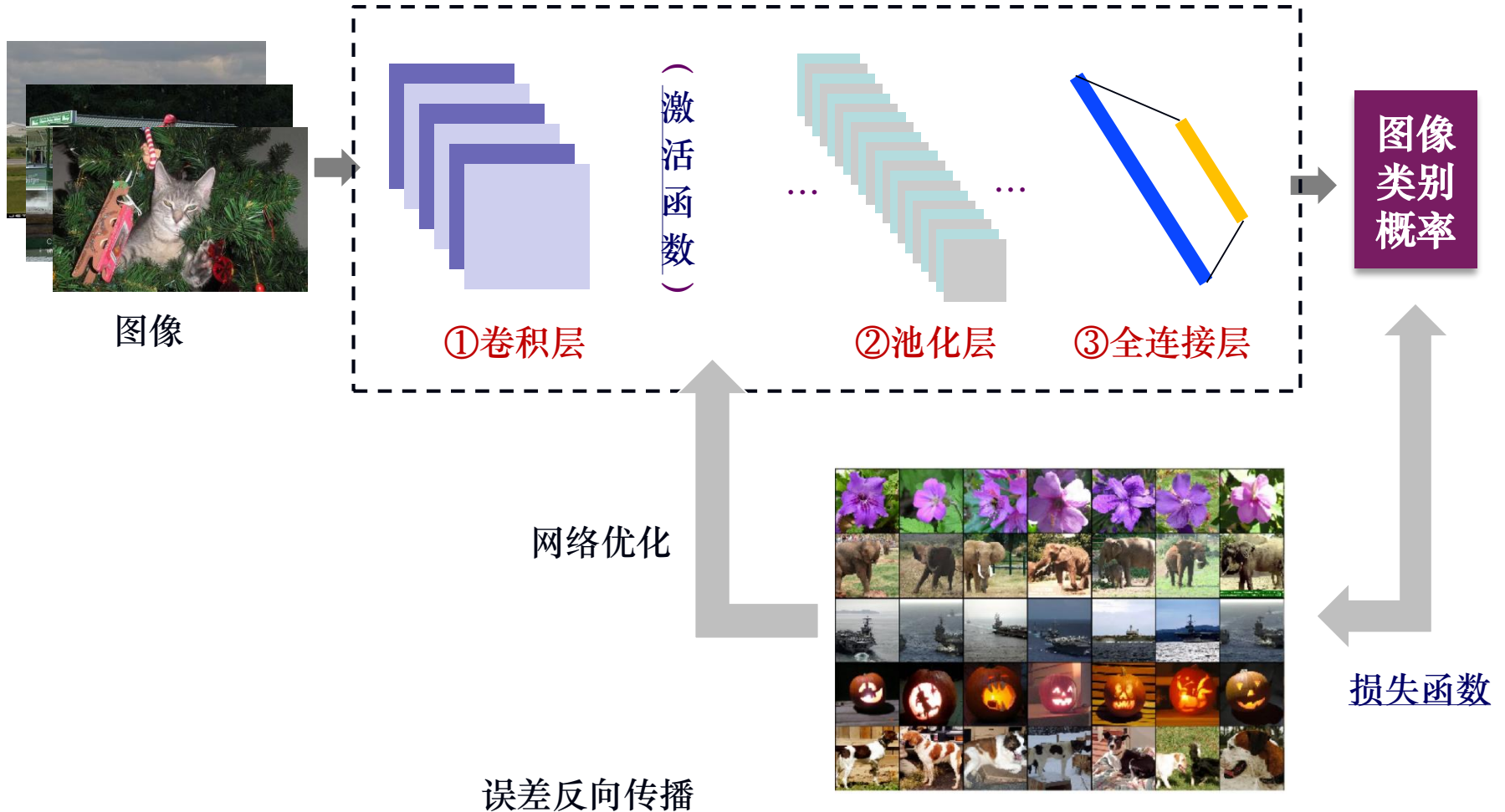
处理数据集

MindSpore提供基于Pipeline的[数据引擎](#)，通过[数据集 \(Dataset\)](#)和[数据变换 \(Transforms\)](#)实现高效的数据预处理。在本教程中，`mindspore.dataset`提供的[数据变换](#)进行预处理。

① 说明

本章节中的示例代码依赖 `download`，可使用命令 `pip install download` 安装。如本文档以Notebook运行时，完成安装后需要重启

卷积神经网络的组成



PyTorch接口：卷积层

➤ 特征图(feature map)尺寸计算

- 输入图像尺寸： $W_1 \times H_1 \times D_1$
- 卷积核尺寸： $F \times F \times D_1$
- 卷积核个数： K
- 卷积核移动步长： S
- 填充pad大小： P

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$

$$D_2 = K$$

Conv2d

<https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

```
CLASS torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1,  
groups=1, bias=True, padding_mode='zeros', device=None, dtype=None) [SOURCE]
```

PyTorch接口：池化层

- 池化层 (Pooling): 使用某一位置的相邻输出的总体统计特征来代替网络在该位置的输出
- 对图像信息进行压缩, **max pooling**, average pooling
 - 输入图像尺寸: $W_1 \times H_1 \times D_1$
 - 池化核尺寸: $F \times F$
 - 池化核移动步长: S

MaxPool2d

<https://pytorch.org/docs/stable/generated/torch.nn.MaxPool2d.html#maxpool2d>

```
CLASS torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False,  
    ceil_mode=False) [SOURCE]
```

PyTorch接口

➤ 全连接层

Linear <https://pytorch.org/docs/stable/generated/torch.nn.Linear.html#torch.nn.Linear>

```
CLASS torch.nn.Linear(in_features, out_features, bias=True, device=None, dtype=None) [SOURCE]
```

➤ 激活函数ReLU

ReLU <https://pytorch.org/docs/stable/generated/torch.nn.ReLU.html>

```
CLASS torch.nn.ReLU(inplace=False) [SOURCE]
```

torch.nn.functional.relu

```
torch.nn.functional.relu(input, inplace=False) → Tensor [SOURCE]
```

LeNet网络模型

model.py



```
import torch.nn as nn
import torch.nn.functional as F
```

5 usages

```
class LeNet(nn.Module):
    def __init__(self):...

    def forward(self, x):...
```

- 源码来源:
<https://github.com/WZMIAOMIAO/deep-learning-for-image-processing>
- 官方教程:
https://pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html

Learn the Basics || Quickstart || Tensors || Datasets & DataLoaders || Transforms || **Build Model** || Autograd || Optimization || Save & Load Model


```
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, 5)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(16, 32, 5)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(32*5*5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

特征提取

分类

```
def forward(self, x):
    x = F.relu(self.conv1(x))      # input(3, 32, 32) output(16, 28, 28)
    x = self.pool1(x)              # output(16, 14, 14)
    x = F.relu(self.conv2(x))      # output(32, 10, 10)
    x = self.pool2(x)              # output(32, 5, 5)
    x = x.view(-1, 32*5*5)          # output(32*5*5)
    x = F.relu(self.fc1(x))         # output(120)
    x = F.relu(self.fc2(x))         # output(84)
    x = self.fc3(x)                # output(10)
    return x
```

前向计算

网络训练

train.py

```
def main():
    transform = transforms.Compose(
        [transforms.ToTensor(),
         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

    # 50000张训练图片
    # 第一次使用时要将download设置为True才会自动去下载数据集
    train_set = torchvision.datasets.CIFAR10(root='./data', train=True,
                                              download=False, transform=transform)
    train_loader = torch.utils.data.DataLoader(train_set, batch_size=36,
                                              shuffle=True, num_workers=0)

    # 10000张验证图片
    # 第一次使用时要将download设置为True才会自动去下载数据集
    val_set = torchvision.datasets.CIFAR10(root='./data', train=False,
                                            download=False, transform=transform)
    val_loader = torch.utils.data.DataLoader(val_set, batch_size=5000,
                                            shuffle=False, num_workers=0)

    val_data_iter = iter(val_loader)
    val_image, val_label = val_data_iter.next()
```

超参数

CIFAR10



- 由60000张32*32彩色图像组成
- 图像有10个类，每个类有6000个图像
- 分别包含有50000个训练图像和10000个测试图像

airplane

automobile

bird

cat

deer

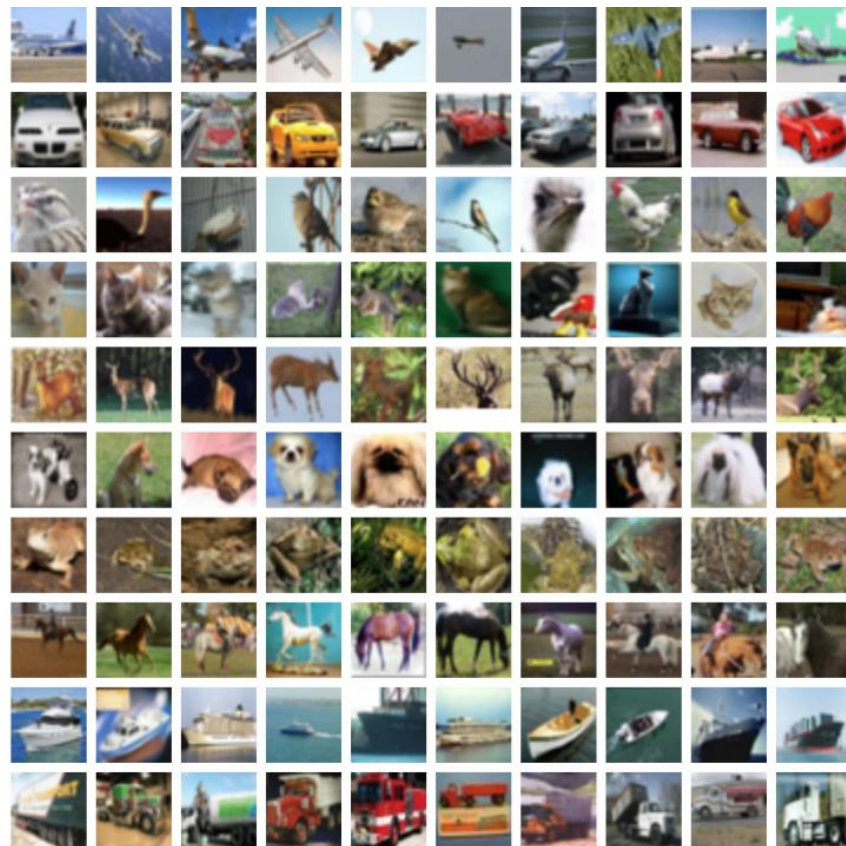
dog

frog

horse

ship

truck



网络训练

train.py



```
net = LeNet()
loss_function = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.001)

for epoch in range(5): # loop over the dataset multiple times

    running_loss = 0.0
    for step, data in enumerate(train_loader, start=0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        ① optimizer.zero_grad()
        # forward + backward + optimize
        ② outputs = net(inputs)
        loss = loss_function(outputs, labels)
        ③ loss.backward()
        ④ optimizer.step()
```

超参数

```
# print statistics
running_loss += loss.item()
if step % 500 == 499:    # print every 500 mini-batches
    with torch.no_grad():
        outputs = net(val_image) # [batch, 10]
        predict_y = torch.max(outputs, dim=1)[1]
        accuracy = torch.eq(predict_y, val_label).sum().item() / val_label.size(0)

    print('[%d, %5d] train_loss: %.3f  test_accuracy: %.3f' %
          (epoch + 1, step + 1, running_loss / 500, accuracy))
    running_loss = 0.0

print('Finished Training')

save_path = './Lenet.pth'
torch.save(net.state_dict(), save_path)
```

```
[1,  500] train_loss: 1.735  test_accuracy: 0.443
[1, 1000] train_loss: 1.431  test_accuracy: 0.506
[2,  500] train_loss: 1.211  test_accuracy: 0.569
[2, 1000] train_loss: 1.158  test_accuracy: 0.593
[3,  500] train_loss: 1.016  test_accuracy: 0.632
[3, 1000] train_loss: 1.010  test_accuracy: 0.635
[4,  500] train_loss: 0.903  test_accuracy: 0.650
[4, 1000] train_loss: 0.918  test_accuracy: 0.659
[5,  500] train_loss: 0.820  test_accuracy: 0.657
[5, 1000] train_loss: 0.837  test_accuracy: 0.686
```

预测/推理

predict.py

```
def main():
    transform = transforms.Compose(
        [transforms.Resize((32, 32)),
         transforms.ToTensor(),
         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

    classes = ('plane', 'car', 'bird', 'cat',
               'deer', 'dog', 'frog', 'horse', 'ship', 'truck')

    net = LeNet()
    net.load_state_dict(torch.load('Lenet.pth'))

    im = Image.open('1.jpg')
    im = transform(im) # [C, H, W]
    im = torch.unsqueeze(im, dim=0) # [N, C, H, W]

    with torch.no_grad():
        outputs = net(im)
        predict = torch.max(outputs, dim=1)[1].numpy()
    print(classes[int(predict)])
```


课程编程讲解说明



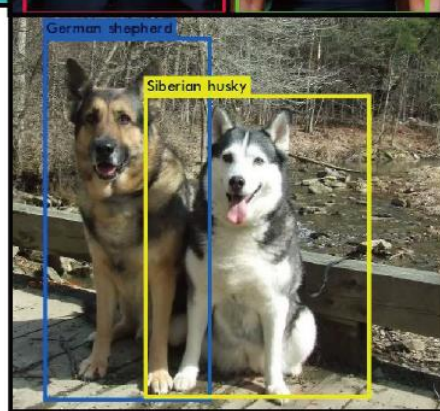
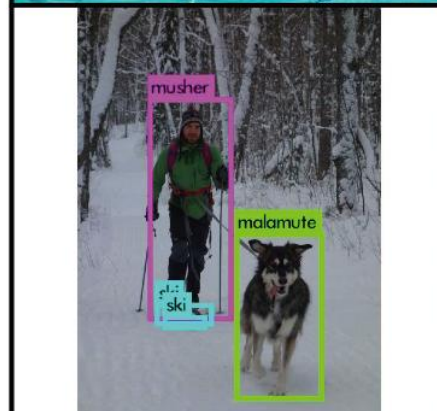
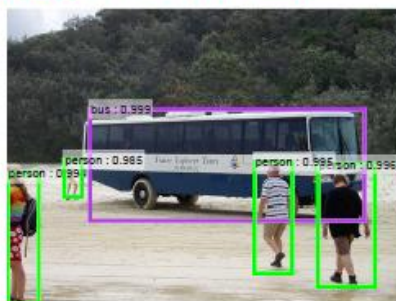
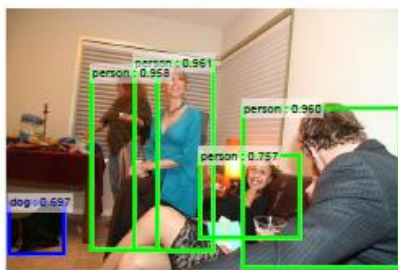
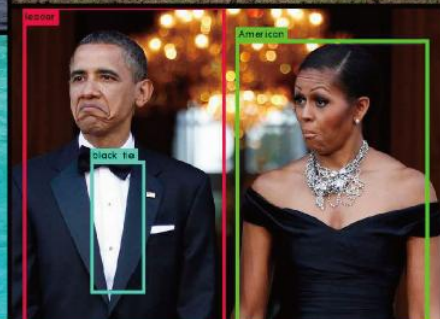
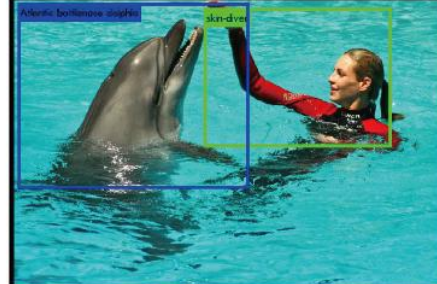
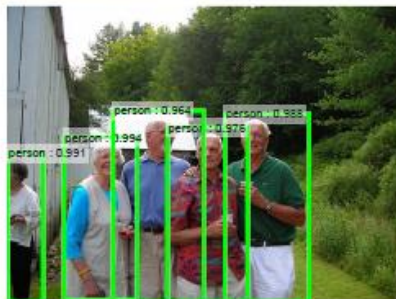
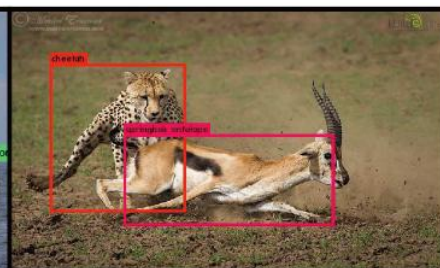
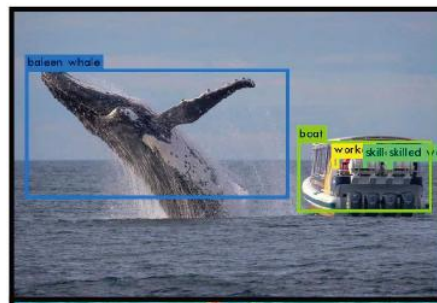
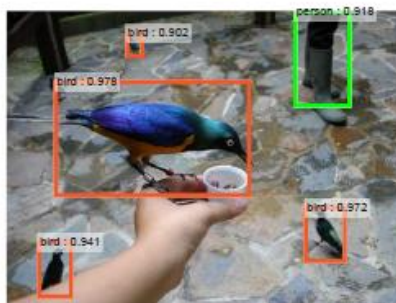
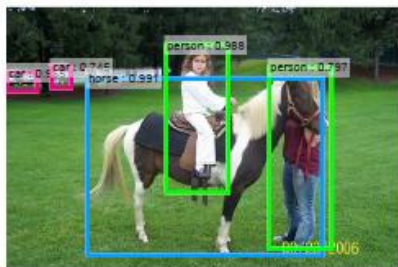
➤ <https://space.bilibili.com/18161609/channel/collectiondetail?sid=48290>

 1.1 卷积神经网络基础 15万 2020-2-25	 1.2 卷积神经网络基础补充 8.1万 2020-2-25	 2.1 pytorch官方demo(Lenet) 15.9万 2020-2-25	 2.2 tensorflow2官方demo 4.5万 2020-2-25	 3.1 AlexNet网络结构详解与花分类数据集下载 11.3万 2020-2-25	 3.2 使用pytorch搭建AlexNet并训练花分类数据集 14万 2020-2-25
 3.3 使用tensorflow2搭建Alexnet 3.5万 2020-2-25	 4.1 VGG网络详解及感受野的计算 7.7万 2020-2-25	 4.2 使用pytorch搭建VGG网络 8.2万 2020-2-25	 4.3 使用tensorflow搭建VGG网络 2.6万 2020-2-25	 5.1 GoogLeNet网络详解 5.7万 2020-2-25	 5.2 使用pytorch搭建GoogLeNet网络 4.7万 2020-2-25
 5.3 使用tensorflow搭建GoogLeNet网络 1.5万 2020-2-25	 6.1 ResNet网络结构, BN以及迁移学习详解 17.7万 2020-2-27	 6.1.2 ResNeXt网络结构 6万 2021-2-5	 6.2 使用pytorch搭建ResNet并基于迁移学习训练 15.9万 2020-2-29	 6.2.2 使用pytorch搭建ResNeXt并基于迁移学习训练 3.3万 2021-2-6	 6.3 使用tensorflow搭建ResNet网络并基于迁移学习的方法进行训练 3.2万 2020-3-3

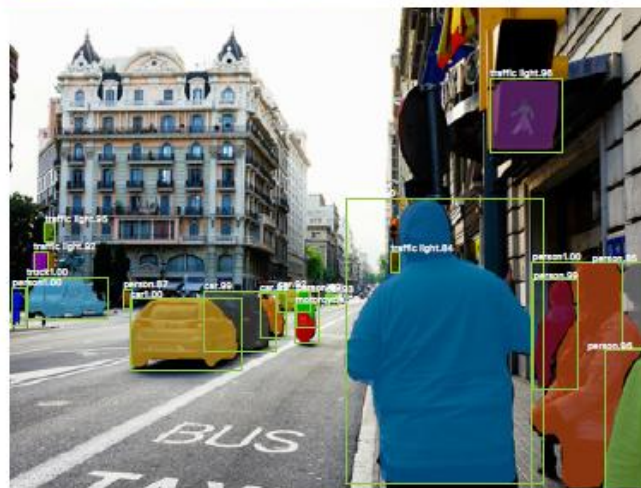
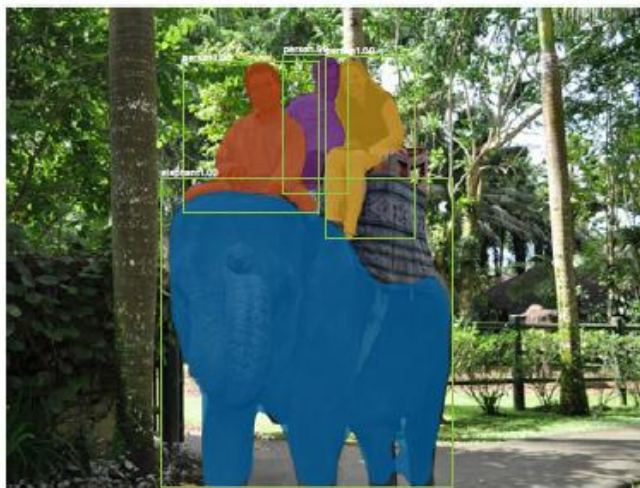
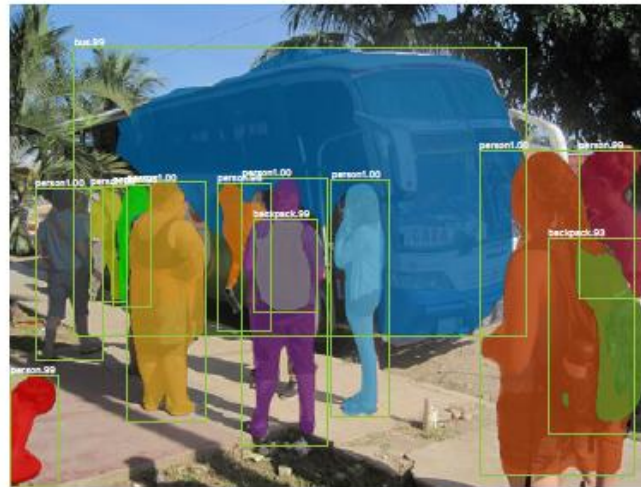
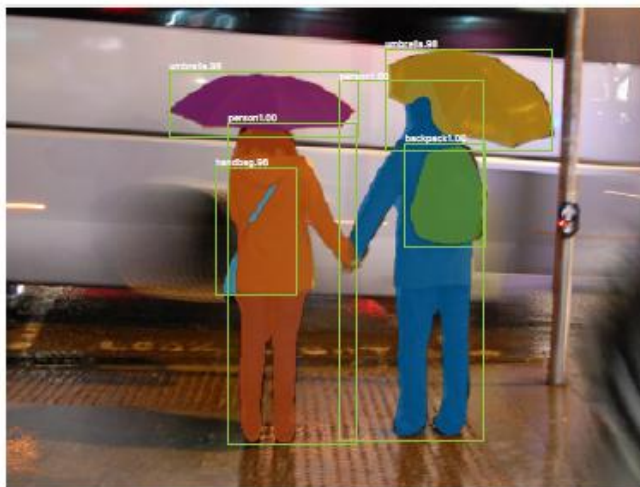
课程编程讲解要求

- 掌握网络结构设计
 - `__init__`、`forward`函数
- 掌握数据集相关操作
- 掌握网络训练流程
 - 设定损失函数、优化器
 - 设定epoch，训练

深度学习视觉应用：目标检测



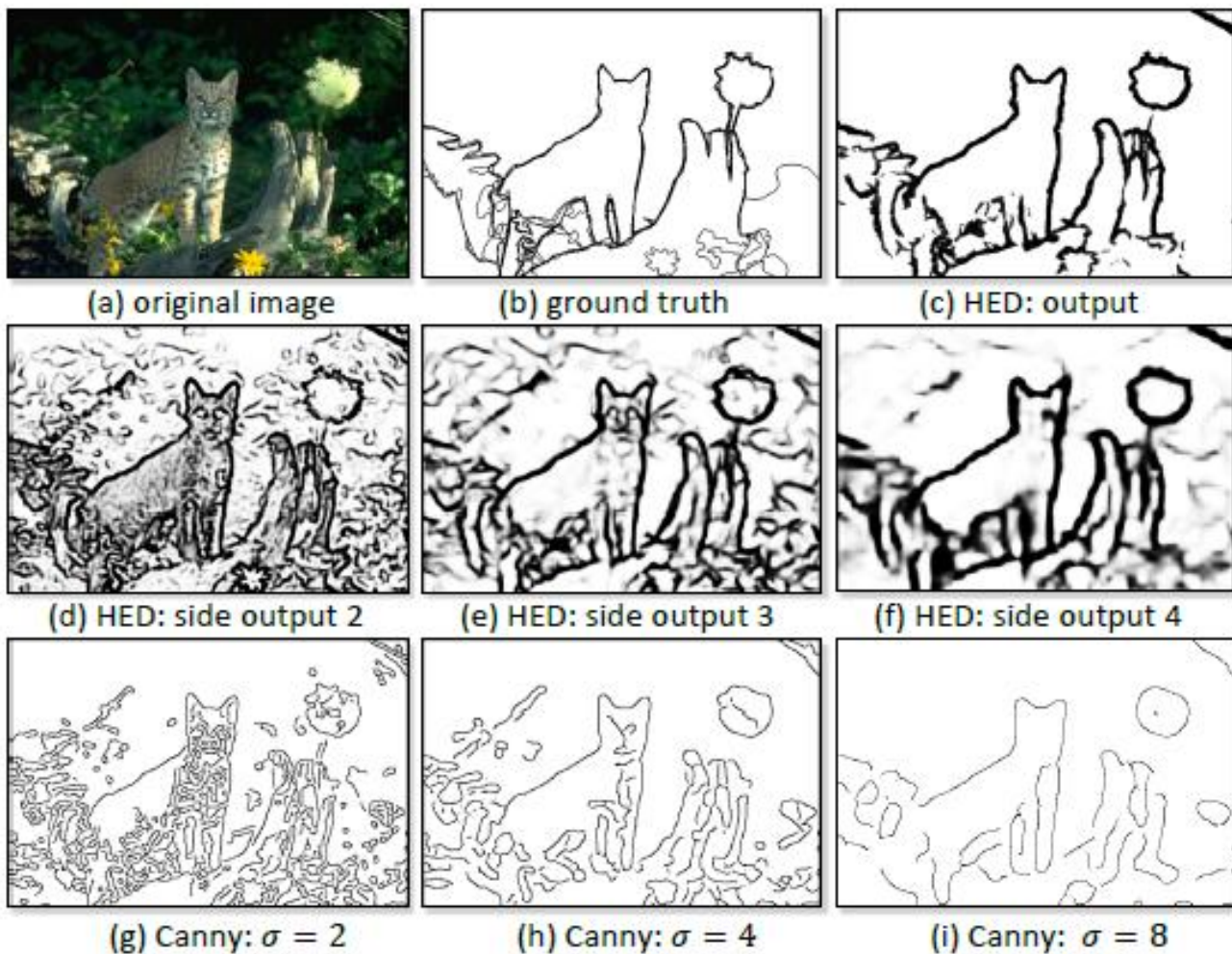
深度学习视觉应用：图像分割



深度学习视觉应用：图像分割



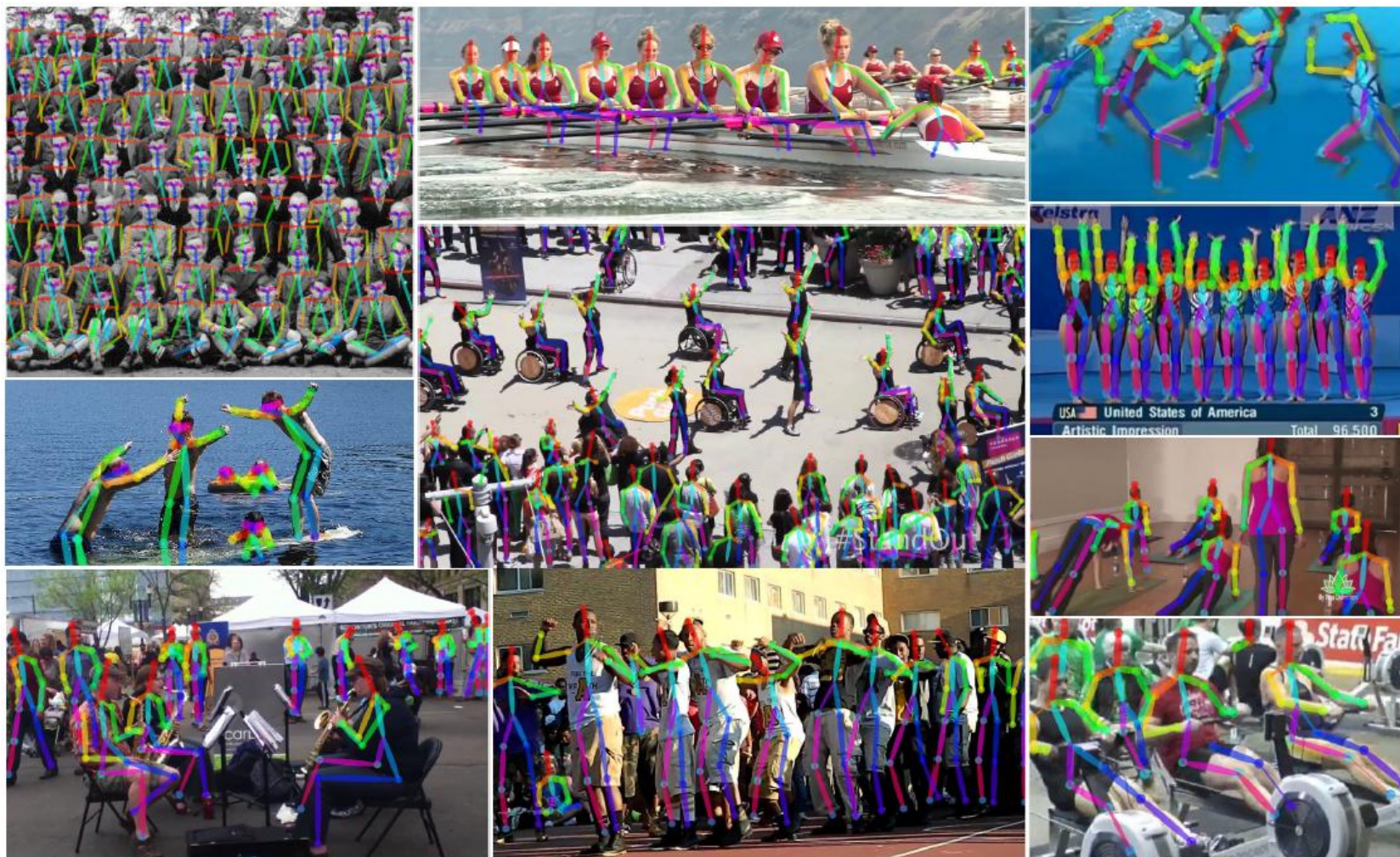
深度学习视觉应用：边缘检测



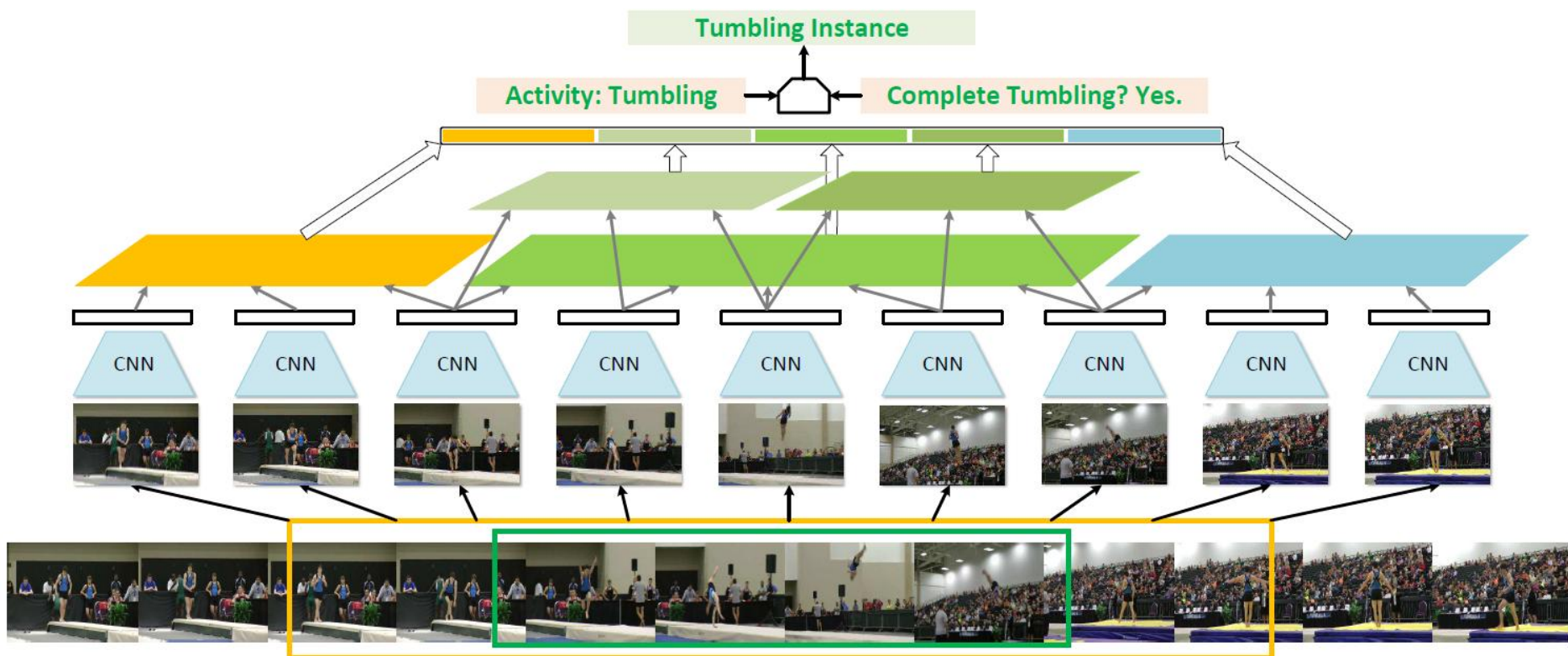
深度学习视觉应用：骨架检测



深度学习视觉应用：姿势识别



深度学习视觉应用：时序动作检测



深度学习视觉应用：图像风格转换



Monet ↔ Photos



Monet → photo

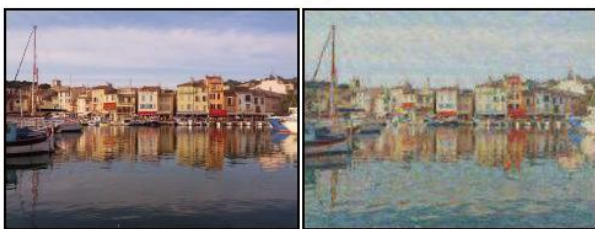


photo → Monet

Zebras ↔ Horses



zebra → horse



horse → zebra

Summer ↔ Winter



summer → winter



winter → summer



Photograph



Monet



Van Gogh



Cezanne

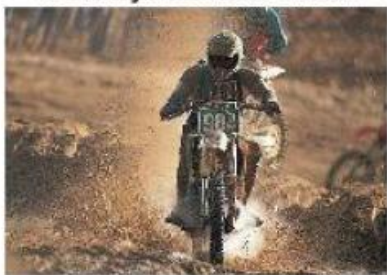


Ukiyo-e

深度学习视觉应用：图像标题生成



A person riding a motorcycle on a dirt road.



Two dogs play in the grass.



A skateboarder does a trick on a ramp.



A dog is jumping to catch a frisbee.



A group of young people playing a game of frisbee.



Two hockey players are fighting over the puck.



A little girl in a pink hat is blowing bubbles.



A refrigerator filled with lots of food and drinks.



A herd of elephants walking across a dry grass field.



A close up of a cat laying on a couch.



A red motorcycle parked on the side of the road.



A yellow school bus parked in a parking lot.



Describes without errors

Describes with minor errors

Somewhat related to the image

Unrelated to the image

深度学习视觉应用：图像生成



Midjourney V7绘画作品



CSDN @阿噜噜小栈

深度学习视觉应用：视频生成



➤ Sora作品



6.5 深度网络中的优化

➤ 学习与纯优化的区别

- 纯优化：最小化目标 J 本身
- 学习：寻找神经网络上的一组参数，它能显著地降低代价函数 $J(\theta)$ ，该代价函数通常包括整个训练集上的性能评估和额外的正则化项

$$J(\theta) = -E_{x,y \sim p_{data}} L(f(x; \theta), y)$$

- 由于数据真实分布未知，通过训练集的经验分布替换真实分布

$$E_{x,y \sim \tilde{p}_{data}} [L(f(x; \theta), y)] = \frac{1}{m} \sum_{i=1}^m L(f(x^{(i)}; \theta), y^{(i)})$$

- 经验风险最小化：最小化平均训练误差，容易过拟合

6.5 深度网络中的优化

➤ 学习与纯优化的区别

- 优化代理损失函数
- 例如，分类器使用对数似然替代函数时，在训练集上的0-1 损失达到0之后，测试集上的0-1损失还能持续。拉开不同类别的距离可以改进分类器鲁棒性
- 训练算法的优化一般不会停止在局部极小点
- 提前终止时代理损失函数仍然有较大的导数，而纯优化终止时导数较小

6.5 深度网络中的优化

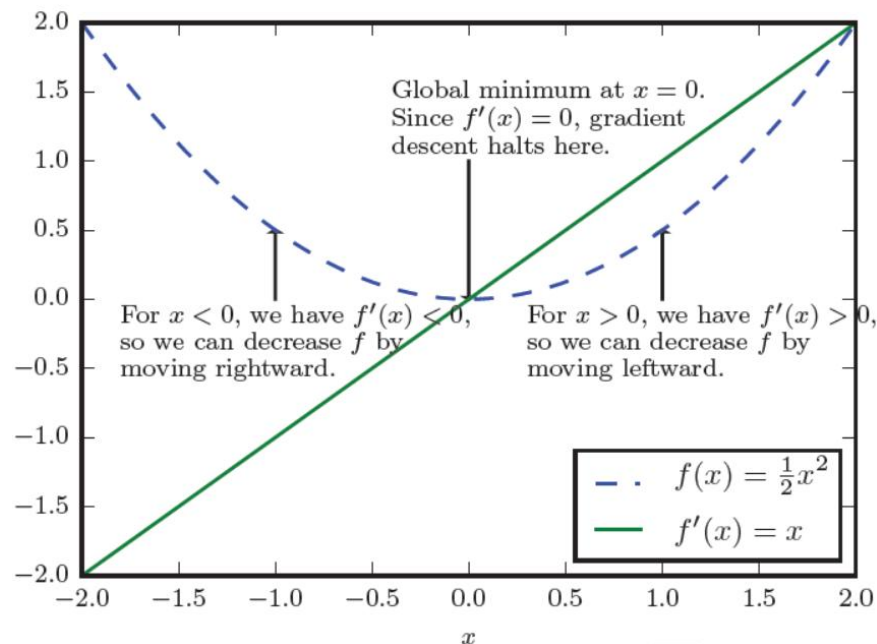
➤ 基于梯度的优化方法

- 对损失函数 (loss function) 或误差函数进行最小化

$$\mathbf{x}^* = \mathbf{argmin} f(\mathbf{x})$$

- 梯度下降：函数的导数 $f'(x)$ 代表 $f(x)$ 在 x 处的斜率， $f(x + \epsilon) \approx f(x) + \epsilon f'(x)$ ，将 x 往导数的反方向移动一小步来减小 $f(x)$

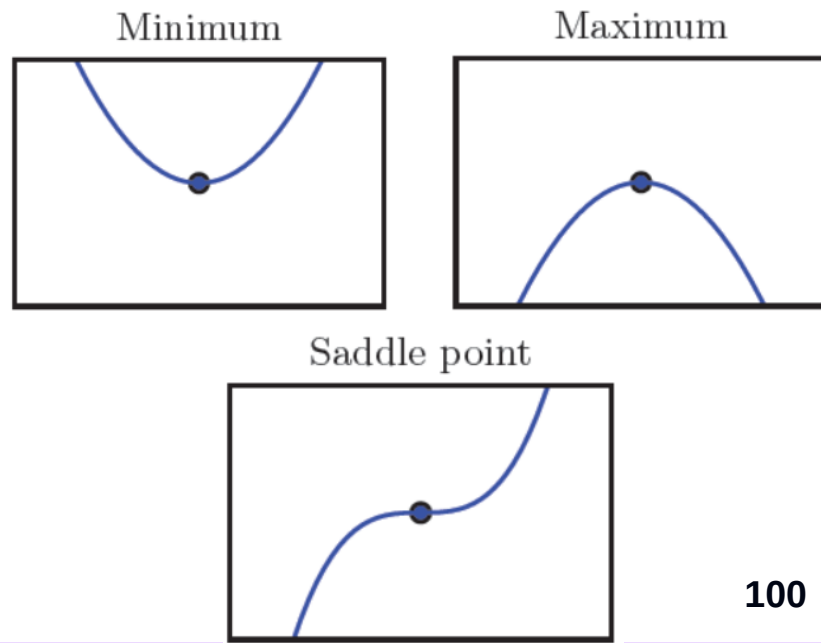
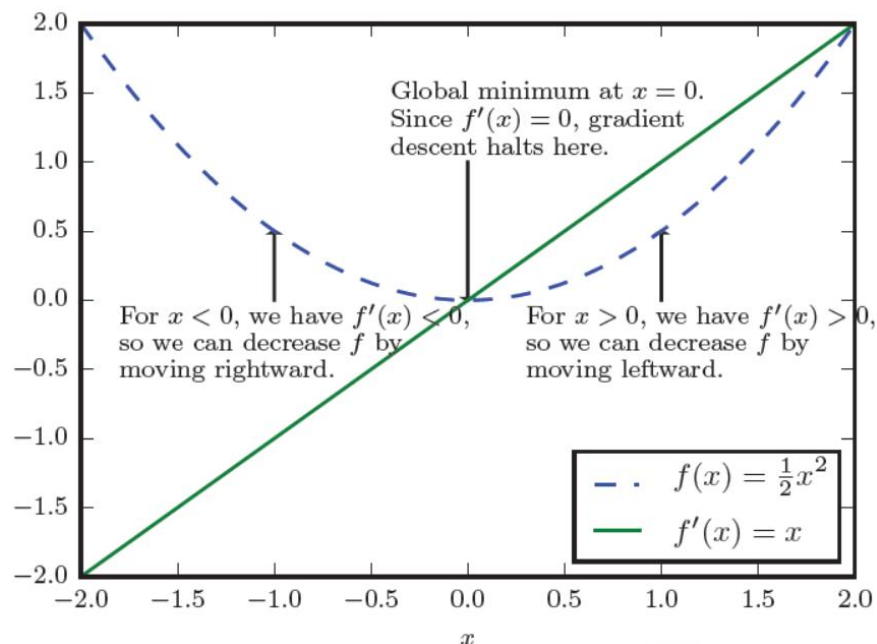
- 临界点/驻点： $f'(x)=0$



6.5 深度网络中的优化

➤ 基于梯度的优化方法

- 局部极小点 (local minimum) : 点的 $f(x)$ 小于所有邻近点, 因此不可能通过移动无穷小的步长来减小 $f(x)$
- 局部极大点 (local maximum) : 点的 $f(x)$ 大于所有邻近点, 因此不可能通过移动无穷小的步长来增大 $f(x)$

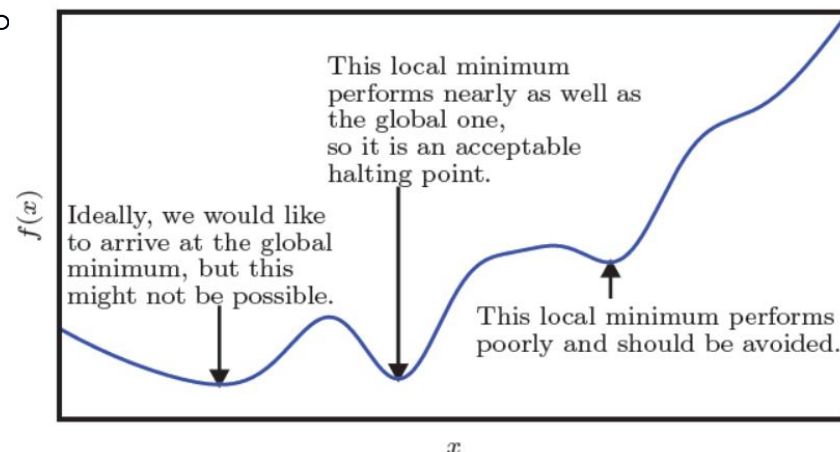


6.5 深度网络中的优化

➤ 基于梯度的优化方法

- 全局最小点 (global minimum) : 使 $f(x)$ 取得绝对的最小值 (相对所有其他值) 的点
- 函数可能只有一个全局最小点或存在多个全局最小点, 还可能不存在不是全局最优的局部极小点
- 深度学习优化难点: 函数可能含有许多不是最优的局部极小点, 或者还有很多处于非常平坦的区域内的鞍点。尤其是当输入是多维的时候。

近似最小化



6.5 深度网络中的优化

➤ 基于梯度的优化方法

- 偏导数 (partial derivative) : 针对多维输入的函数, $\frac{\partial}{\partial x_i} f(\mathbf{x})$ 衡量点 $\mathbf{x}$ 处只有 x_i 增加时 $f(\mathbf{x})$ 如何变化
- 梯度 (gradient) : 相对一个向量求导的导数, f 的导数是包含所有偏导数的向量, 记为 $\nabla_{\mathbf{x}} f(\mathbf{x})$ 。梯度的第 i 个元素是 f 关于 x_i 的偏导数
- 在多维情况下, 临界点是梯度中所有元素都为零的点
- $\mathbf{u}$ (单位向量) 方向的方向导数 (directional derivative) : 函数 f 在 $\mathbf{u}$ 方向的斜率, 即 $f(\mathbf{x} + \alpha \mathbf{u})$ 关于 α 的导数 (在 $\alpha = 0$ 时取得)。使用链式法则, 我们可以看到当 $\alpha = 0$ 时, $\frac{\partial}{\partial \alpha} f(\mathbf{x} + \alpha \mathbf{u}) = \mathbf{u}^T \nabla_{\mathbf{x}} f(\mathbf{x})$

6.5 深度网络中的优化

➤ 基于梯度的优化方法

- 为了最小化 f ，希望找到使 f 下降得最快的方向。计算方向导数：

$$\min_{u, u^T u = 1} \mathbf{u}^T \nabla_x f(\mathbf{x}) = \min_{u, u^T u = 1} \|\mathbf{u}\|_2 \|\nabla_x f(\mathbf{x})\|_2 \cos \theta \rightarrow \min_u \cos \theta$$

- θ 是 $\mathbf{u}$ 与梯度的夹角， $\mathbf{u}$ 与梯度方向相反时取得最小
- 最速下降法(method of steepest descent) 或梯度下降 (gradient descent)：负梯度方向上移动可以减小 f ，最速下降建议新的点为

$$\mathbf{x}' = \mathbf{x} - \epsilon \nabla_x f(\mathbf{x})$$

- ϵ 是学习率 (learning rate)，确定步长大小的正标量。可以通过几种不同的方式选择 ϵ ，普遍方式是选择一个小常数

6.5 深度网络中的优化

➤ 基于梯度的优化方法

- 最速下降在梯度的每一个元素为零时收敛（或在实践中，很接近零时）。在某些情况下能够避免运行该迭代算法，并通过解方程 $\nabla_x f(x) = 0$ 直接跳到临界点。
- 虽然梯度下降被限制在连续空间中的优化问题，但不断向更好的情况移动一小步（即近似最佳的小移动）的一般概念可以推广到离散空间。递增带有离散参数的目标函数被称为爬山（hill climbing）算法(Russel and Norvig, 2003)。

6.5 深度网络中的优化

➤ 基于梯度的优化方法

- 梯度之上：Jacobian 和 Hessian 矩阵
- Jacobian(雅克比)矩阵：计算输入和输出都为向量的函数的所有偏导数，包含所有这样的偏导数的矩阵被称为 Jacobian 矩阵
- $f: \mathbb{R}^m \rightarrow \mathbb{R}^n$, f 的 Jacobian 矩阵 $J \in \mathbb{R}^{m \times n}$ 定义为

$$J_{i,j} = \frac{\partial}{\partial x_j} f(x)_i$$

- 二阶导数：导数的导数 $\frac{\partial^2}{\partial x_i \partial x_j} f$, 指示一阶导数将如何随着输入的变化而改变，表示只基于梯度信息的梯度下降步骤是否会产生如预期的改善

6.5 深度网络中的优化

➤ 基于梯度的优化方法

- Hessian矩阵：函数具有多维输入时，将二阶导数合并成一个矩阵， $H(f)(\mathbf{x})$ 定义为

$$H(f)(\mathbf{x})_{i,j} = \frac{\partial^2}{\partial x_i \partial x_j} f(\mathbf{x})$$

- Hessian等价于梯度的Jacobian矩阵
- 微分算子在任何二阶偏导连续点处可交换

$$\frac{\partial^2}{\partial x_i \partial x_j} f(\mathbf{x}) = \frac{\partial^2}{\partial x_j \partial x_i} f(\mathbf{x})$$

6.5 深度网络中的优化

➤ 基于梯度的优化方法

- 在当前点 $\mathbf{x}^{(0)}$ 处作函数 $f(\mathbf{x})$ 的近似二阶泰勒级数:

$$f(\mathbf{x}) \approx f(\mathbf{x}^{(0)}) + (\mathbf{x} - \mathbf{x}^{(0)})^T \mathbf{g} + \frac{1}{2} (\mathbf{x} - \mathbf{x}^{(0)})^T \mathbf{H} (\mathbf{x} - \mathbf{x}^{(0)})$$

- 其中 $\mathbf{g}$ 是梯度, $\mathbf{H}$ 是 $\mathbf{x}^{(0)}$ 点的Hessian
- 使用学习率 ϵ , 那么新的点 $\mathbf{x}$ 将会是 $\mathbf{x}^{(0)} - \epsilon \mathbf{g}$ 。代入上述的近似, 可得

$$f(\mathbf{x}^{(0)} - \epsilon \mathbf{g}) \approx f(\mathbf{x}^{(0)}) - \epsilon \mathbf{g}^T \mathbf{g} + \frac{1}{2} \epsilon^2 \mathbf{g}^T \mathbf{H} \mathbf{g}$$

- 公式右侧3项分别表示: 函数的原始值、函数斜率导致的预期改善、函数曲率导致的校正

6.5 深度网络中的优化

➤ 基于梯度的优化方法

- 使用Hessian矩阵指导搜索，最简单的方法是牛顿法（Newton's method）

- 牛顿法基于一个二阶泰勒展开来近似 $x^{(0)}$ 附近的 $f(x)$:

$$f(x) \approx f(x^{(0)}) + (x - x^{(0)})^T \nabla_x f(x^{(0)}) + \frac{1}{2} (x - x^{(0)})^T H(f)(x^{(0)}) (x - x^{(0)})$$

- 函数的临界点

$$x^* = x^{(0)} - H(f)(x^{(0)})^{-1} \nabla_x f(x^{(0)})$$

- 当 f 是一个正定二次函数时，牛顿法只要应用一次式子就能直接跳到函数的最小点；否则多次迭代

6.5 深度网络中的优化

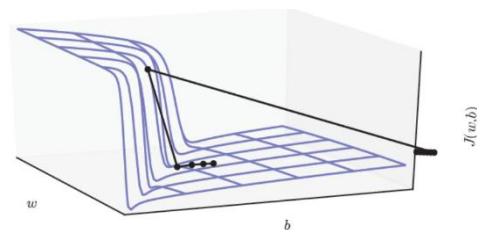
➤ 基于梯度的优化方法

- 仅使用梯度信息的优化算法被称为一阶优化算法(first-order optimization algorithms)，如梯度下降。
- 使用Hessian 矩阵的优化算法被称为二阶最优化算法(second-order optimization algorithms)，如牛顿法。
- 最成功的特定优化领域或许是凸优化 (Convex optimization) 。
- 凸优化通过更强的限制提供更多的保证，凸优化算法只对凸函数适用，即Hessian 处处半正定的函数。
- 深度学习中的大多数问题都难以表示成凸优化的形式。

6.5 深度网络中的优化

➤ 神经网络优化中的挑战

- 长期依赖
- 由于变深的结构使模型丧失了学习到先前信息的能力，让优化变得极其困难
- 假设计算图中反复与矩阵 W 相乘，那么 t 步后，相当于乘以 W^t 。假设 W 有特征值分解 $W = V\text{diag}(\lambda) V^1$ 。在这种情况下，很容易看出 $W^t = (V\text{diag}(\lambda) V^1)^t = V\text{diag}(\lambda)^t V^1$ 。当特征值不在1 附近时，若在量级上大于1 则会爆炸；若小于1 时则会消失。
- 梯度消失：不知道参数朝哪个方向前进
- 梯度爆炸：学习不稳定（悬崖结构）



6.5 深度网络中的优化

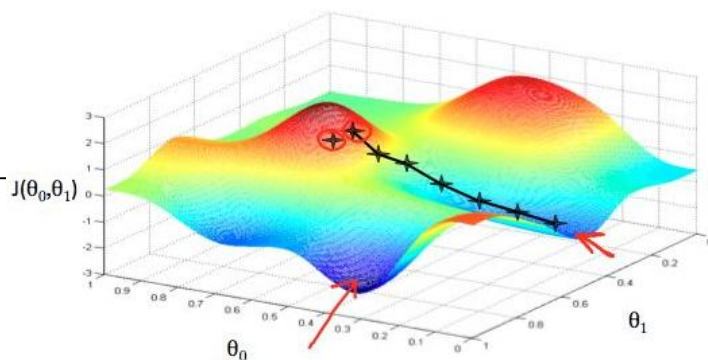
➤ 基本算法

1、随机梯度下降 (SGD)

- 方法：按照数据生成分布抽取 m 个小批量（独立同分布的）样本，通过计算它们梯度均值，我们可以得到梯度的无偏估计

$$\theta \leftarrow \theta - \epsilon \frac{1}{m} \nabla_{\theta} \left(\sum_{i=1}^m L(f(x^{(i)}; \theta), y^{(i)}) \right)$$

- 关键参数：学习率 ϵ ，可通过试验和误差来选取
- 性质：每一步更新的计算时间不依赖训练样本的多少，即使非常大，也能收敛
- 收敛率：衡量额外误差 $J(\theta) - \min_{\theta} J(\theta)$ ，即当前代价函数超出最低可能代价的量
- 缺点：学习过程有时会很慢



算法 8.1 随机梯度下降 (SGD) 在第 k 个训练迭代的更新

Require: 学习率 ϵ_k

Require: 初始参数 θ

while 停止准则未满足 **do**

从训练集中采包含 m 个样本 $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ 的小批量, 其中 $\mathbf{x}^{(i)}$ 对应目标为 $\mathbf{y}^{(i)}$ 。

计算梯度估计: $\hat{\mathbf{g}} \leftarrow +\frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

应用更新: $\theta \leftarrow \theta - \epsilon \hat{\mathbf{g}}$

end while

- 关键参数: 学习率 ϵ , 可通过试验和误差来选取
- 性质: 每一步更新的计算时间不依赖训练样本的多少, 即使非常大, 也能收敛
- 收敛率: 衡量额外误差 $J(\theta) - \min_{\theta} J(\theta)$, 即当前代价函数超出最低可能代价的量
- 缺点: 学习过程有时会很慢

6.5 深度网络中的优化

➤ 基本算法

2、动量方法

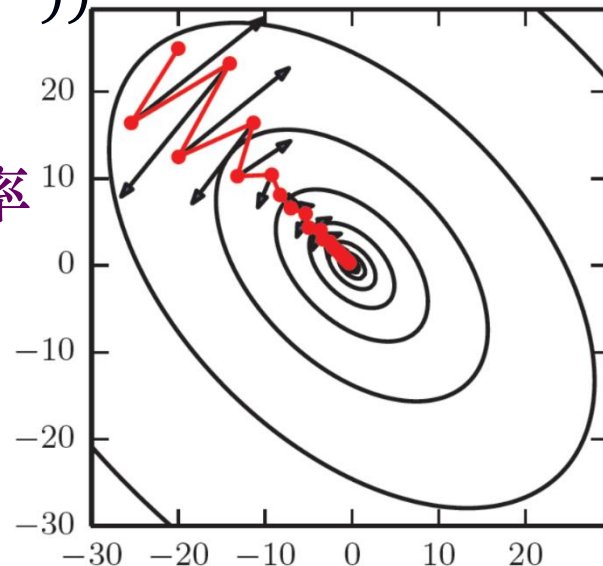
- 目的:旨在加速学习，特别是处理高曲率、小但一致的梯度，或是带噪声的梯度。动量算法积累了之前梯度指数级衰减的移动平均，并且继续沿该方向移动。

- $v \leftarrow \alpha v - \epsilon \frac{1}{m} \nabla_{\theta} (\sum_{i=1}^m L(f(x^{(i)}; \theta), y^{(i)}))$

- $\theta \leftarrow \theta + v$

- v 代表参数在参数空间移动方向和速率

- 速度被设为负梯度的指数衰减平均

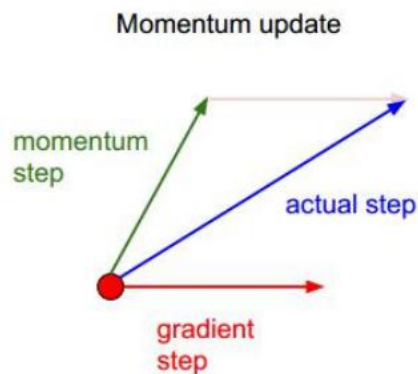


6.5 深度网络中的优化

➤ 基本算法

3、Nesterov 动量

- $v \leftarrow \alpha v - \epsilon \frac{1}{m} \nabla_{\theta} (\sum_{i=1}^m L(f(x^{(i)}; \theta + \alpha v), y^{(i)}))$
- $\theta \leftarrow \theta + v$
- 和标准动量之间的区别体现在梯度计算，往标准动量方法中添加了一个校正因子



6.5 深度网络中的优化

➤ 参数初始化策略

- 训练算法通常是迭代的，因此要求使用者指定一些开始迭代的初始点
- 初始点能够决定算法是否收敛，有些初始点十分不稳定
- 当学习收敛时，初始点可以决定学习收敛得多快，以及是否收敛到一个代价高或低的点
- 初始参数需要在不同单元间“破坏对称性”，更大的初始权重具有更强的破坏对称性的作用，有助于避免冗余的单元
- 初始化模型：高斯或均匀分布中随机产生，或者启发式方法，如随机正交矩阵

6.5 深度网络中的优化

➤ 自适应学习率算法

- Delta-bar-delta 算法(Jacobs, 1988) 是一个早期的在训练时适应模型参数各自学习率的启发式方法。该方法基于一个很简单的想法，如果损失对于某个给定模型参数的偏导保持相同的符号，那么学习率应该增加。如果对于该参数的偏导变化了符号，那么学习率应减小。当然，这种方法只能应用于全批量优化中。

6.5 深度网络中的优化

➤ 自适应学习率算法：AdaGrad

- 缩放参数反比于其所有梯度历史平方值总和的平方根。具有损失最大偏导的参数有一个快速下降的学习率，而具有小偏导的参数在学习率上有相对较小的下降。净效果是在参数空间中更为平缓的倾斜方向会取得更大进步
- 从训练开始时积累梯度平方会导致有效学习率过早过量减小
- 超参数：全局学习率 ε
- 计算梯度： $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} (\sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)}))$
- 累计平方梯度： $\mathbf{r} \leftarrow \mathbf{r} + \mathbf{g} \odot \mathbf{g}$
- 计算参数更新： $\Delta \theta \leftarrow -\frac{\varepsilon}{\delta + \sqrt{\mathbf{r}}} \odot \mathbf{g}$ (逐元素应用)
- 应用更新： $\theta \leftarrow \theta + \Delta \theta$

6.5 深度网络中的优化

➤ 自适应学习率算法：RMSProp

- 超参数：全局学习率 ε ，衰减速率 ρ
- 计算梯度： $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} (\sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)}))$
- 累计平方梯度： $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$
- 计算参数更新： $\Delta \theta \leftarrow -\frac{\varepsilon}{\delta + \sqrt{\mathbf{r}}} \odot \mathbf{g}$ （逐元素应用）
- 应用更新： $\theta \leftarrow \theta + \Delta \theta$
- 修改AdaGrad以在非凸设定下效果更好，**改变梯度积累为指数加权的移动平均**。使用指数衰减平均以丢弃遥远过去的历史，使其能够在找到凸碗状结构后快速收敛
- 超参数用来控制移动平均的长度范围
- 一种有效且实用的深度神经网络优化算法

6.5 深度网络中的优化

- 自适应学习率算法：使用Nesterov动量的RMSProp
 - 超参数：全局学习率 ε ，衰减速率 ρ ，动量系数 α
 - 计算临时更新： $\tilde{\theta} \leftarrow \theta + \alpha v$
 - 计算梯度： $g \leftarrow \frac{1}{m} \nabla_{\theta} (\sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)}))$
 - 累计平方梯度： $r \leftarrow \rho r + (1 - \rho) g \odot g$
 - 计算速度更新： $v \leftarrow \alpha v - \frac{\varepsilon}{\delta + \sqrt{r}} \odot g$ （逐元素应用）
 - 应用更新： $\theta \leftarrow \theta + v$

6.5 深度网络中的优化

- 自适应学习率算法：Adam (adaptive moments)
 - 对超参数的选择相当鲁棒
 - 动量并入了梯度一阶矩（指数加权）的估计，将动量应用于缩放后的梯度
 - 偏置修正，修正从原点初始化的一阶矩（动量项）和（非中心的）二阶矩的估计

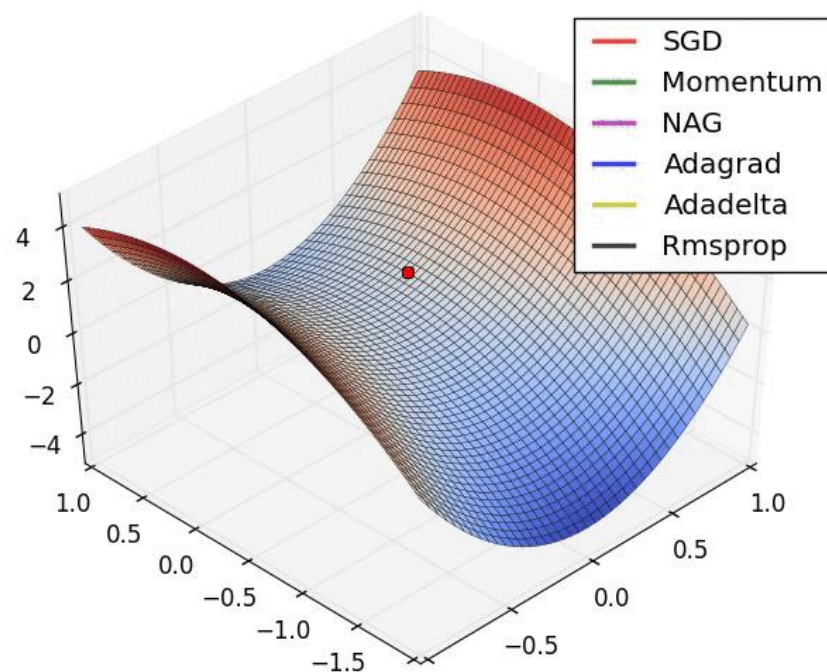
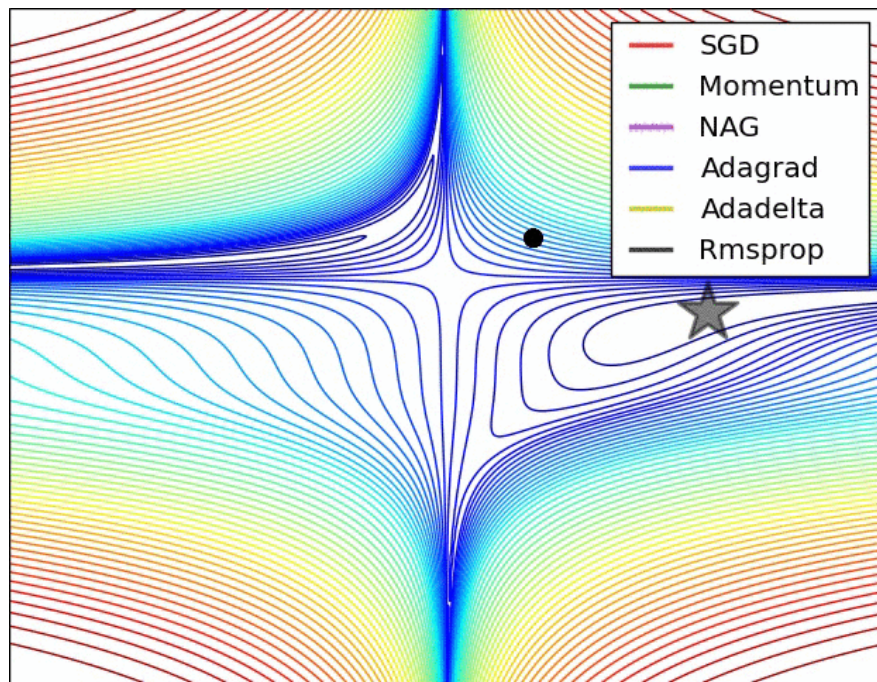
6.5 深度网络中的优化

➤ 自适应学习率算法：Adam (adaptive moments)

- 超参数：步长 ϵ (默认：0.001)，衰减速率 ρ_1, ρ_2 (默认：0.9, 0.999)
- 初始化 $t = 0$
- 计算梯度： $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} (\sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)}))$
- $t = t + 1$
- 更新有偏一阶矩估计： $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$
- 更新有偏二阶矩估计： $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$
- 修正一阶矩、二阶矩偏差： $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t} \quad \hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$
- 计算参数更新： $\Delta \theta \leftarrow -\epsilon \frac{\hat{\mathbf{s}}}{\delta + \sqrt{\hat{\mathbf{r}}}}$
- 应用更新： $\theta \leftarrow \theta + \Delta \theta$

6.5 深度网络中的优化

➤ 自适应学习率算法



6.5 深度网络中的优化

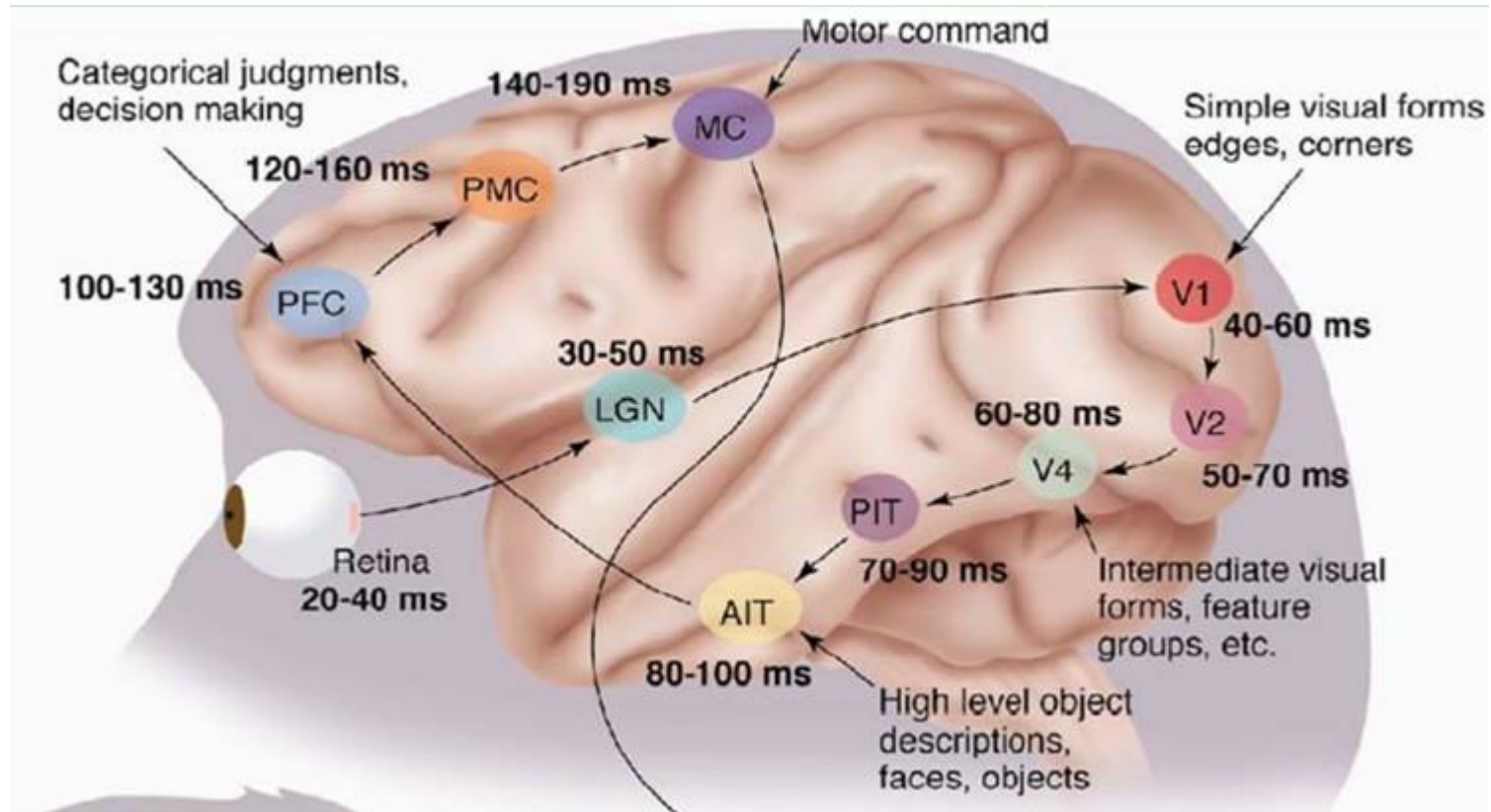
➤ 选择正确的优化算法

- 许多优化算法在大量学习任务上极具价值的比较。虽然结果表明，具有自适应学习率（以RMSProp 和 AdaDelta 为代表）的算法族表现得相当鲁棒，不分伯仲，但没有哪个算法能脱颖而出。
- 最流行并且使用很高的优化算法包括：SGD、具动量的SGD、RMSProp、具动量的RMSProp、AdaDelta 和 Adam。此时，选择哪一个算法似乎主要取决于使用者对算法的熟悉程度（以便调节超参数）

6.6 卷积网络的神经科学基础

- 为了确定关于哺乳动物视觉系统如何工作的许多最基本的事实，神经生理学家David Hubel 和Torsten Wiesel 合作多年(Hubel and Wiesel, 1959, 1962, 1968)
- 对当代深度学习模型有最大影响的是基于记录猫的单个神经元的活动
- 他们观察了猫的脑内神经元如何响应投影在猫前面屏幕上精确位置的图像。发现处于视觉系统较为前面的神经元对非常特定的光模式（例如精确定向的条纹）反应最强烈，但对其他模式几乎完全没有反应

6.6 卷积网络的神经科学基础



6.6 卷积网络的神经科学基础

- 视觉被称为V1 的大脑的一部分，也称为初级视觉皮层（primary visual cortex）
- V1 是大脑对视觉输入开始执行显著高级处理的第一个区域。图像是由光到达眼睛并刺激视网膜（眼睛后部的光敏组织）形成的
- 视网膜中的神经元对图像执行一些简单的预处理，但是基本不改变它被表示的方式
- 图像通过视神经和称为外侧膝状核的脑部区域。这些解剖区域的主要作用是仅仅将信号从眼睛传递到位于头后部的V1

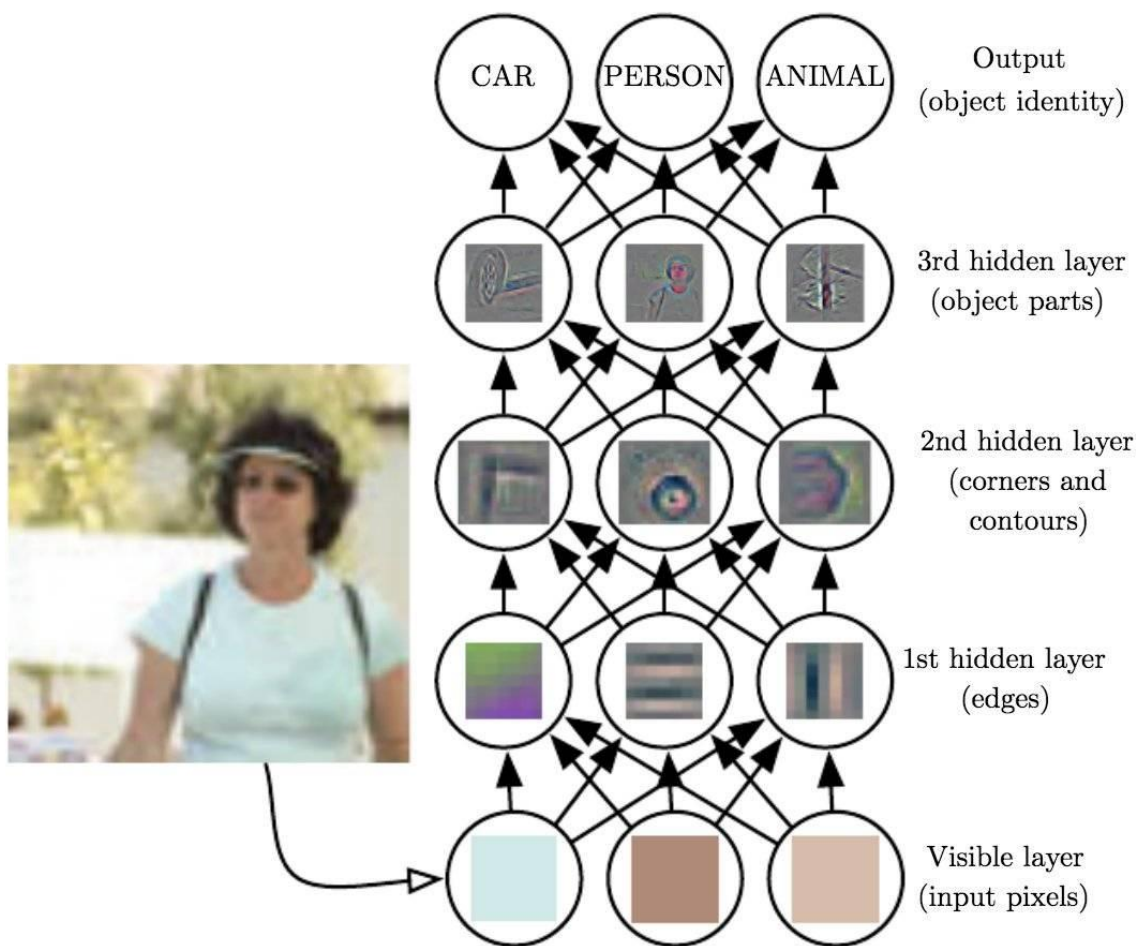
6.6 卷积网络的神经科学基础

- 卷积网络层被设计为描述V1的三个性质：
 - V1 可以进行空间映射。具有二维结构来反映视网膜中的图像结构。卷积网络用二维映射定义特征
 - V1 包含许多简单细胞。简单细胞的活动在某种程度上可以概括为在一个小的空间位置感受野内的图像的线性函数。卷积网络的检测器单元模拟这些性质。
 - V1 包括许多复杂细胞，响应由简单细胞检测的那些特征，且对于特征的位置微小偏移具有不变性。这启发了卷积网络的池化单元。对于照明中的一些变化也是不变的，不能简单地通过在空间位置上池化来刻画。这些不变性激发了卷积网络中的一些跨通道池化策略，例如maxout 单元(Goodfellow *et al.*, 2013b)。

6.6 卷积网络的神经科学基础

区别

1. 人眼大部分都是低分辨率的，卷积网络接收全分辨率图像（注意力机制）
2. 人类视觉集成其他感觉，如听觉
3. 人类视觉系统不仅用于识别对象，能够理解整个场景
4. V1等简单大脑区域也瘦到来自较高级别的反馈的严重影响
5. 前馈IT放电频率刻画了卷积网络特征的很多相同信息，但仍不清楚如何计算



6.7 实践

- 如何针对具体应用挑选一个合适的算法以及如何监控，并根据实验反馈改进学习系统
- 实践设计流程：
 - • 确定目标——使用什么样的误差度量，并为此误差度量指定目标值。这些目标和误差度量取决于该应用旨在解决的问题。
 - • 尽快建立一个端到端的的工作流程，包括估计合适的性能度量。
 - • 搭建系统，并确定性能瓶颈。检查哪个部分的性能差于预期，以及是否是因为过拟合、欠拟合，或者数据或软件缺陷造成的。
 - • 根据具体观察反复地进行增量式的改动，如收集新数据、调整超参数或改进算法。

6.7 实践

- 街景地址号码转录系统(Goodfellow *et al.*, 2014d)
- 目标是将建筑物添加到谷歌地图
 - 街景车拍摄建筑物，并记录与每张建筑照片相关的GPS坐标
 - 卷积网络识别每张照片上的地址号码，由谷歌地图数据库在正确的位置添加该地址

6.7 实践

➤ 1. 确定性能度量和目标：性能度量

- 用什么误差度量？了解大概能得到什么级别的目标性能
 - 不可能实现绝对零误差。因为输入特征可能无法包含输出变量的完整信息，或是因为系统可能本质上是随机的
 - 受限于有限的训练数据。确定进一步减少误差的价值，并与收集更多数据的成本做权衡
- 如何确定合理的性能期望？
 - 学术界，通常我们可以根据先前公布的基准结果来估计预期错误率
 - 现实世界中，一个应用的错误率有必要是安全的、具有成本效益的或吸引消费者的

6.7 实践

➤ 1. 确定性能度量和目标：度量的选择

- 性能度量通常不同于训练模型的代价函数
- 度量一个系统的准确率或错误率
- 度量精度 (precision) 和召回率 (recall) 。精度是模型报告的检测是正确的比率，而召回率则是真实事件被检测到的比率。画PR 曲线 (PR curve) ， y 轴表示精度， x 轴表示召回率，将精度 p 和召回率 r 转换为F 分数F-score)

$$F = \frac{2pr}{p + r}$$

- 报告PR曲线下的总面积

6.7 实践

➤ 1. 确定性能度量和目标：度量的选择

- 街景转录系统中，如果地图是不精确的，那么地图的价值会严重下降。因此只在转录正确的情况下添加地址十分重要
- 性能度量是覆盖（coverage）：机器学习系统能够产生响应的样本所占的比率，权衡覆盖和精度。一个系统可以通过拒绝处理任意样本的方式来达到100%的精度，但是覆盖降到了0%。对于街景任务，该项目的目标是达到人类级别的转录精度，同时保持95%的覆盖。在这项任务中，人类级别的性能是98%的精度。

6.7 实践

- 1. 确定性能度量 and 目标
- 2. 建立一个合理的端到端的系统
 - 根据数据的结构选择一类合适的模型：如果项目是以固定大小的向量作为输入的监督学习，那么可以使用全连接的前馈网络。如果输入有已知的拓扑结构（例如，输入是图像），那么可以使用卷积网络。在这些情况下，刚开始可以使用某些分段线性单元（ReLU 或者其扩展，如Leaky ReLU、PReLU 和maxout）。如果输入或输出是一个序列，可以使用门控循环网络（LSTM或GRU）
 - 优化算法：具有衰减学习率以及动量的SGD或者Adam 算法批标准化对优化性能有着显著的影响，特别是对卷积网络和具有sigmoid 非线性函数的网络而言

6.7 实践

- 1. 确定性能度量 and 目标
- 2. 建立一个合理的端到端的系统
 - 包含一些温和的正则化。提前终止也被普遍采用。Dropout 也是一个很容易实现，且兼容很多模型和训练算法的出色正则化项。批标准化有时也能降低泛化误差，此时可以省略Dropout 步骤，因为用于标准化变量的统计量估计本身就存在噪声。
 - 复制已有相似任务的模型，例如通常会使用在ImageNet上训练好的卷积网络的特征来解决其他计算机视觉任务 (Girshick *et al.*, 2015)

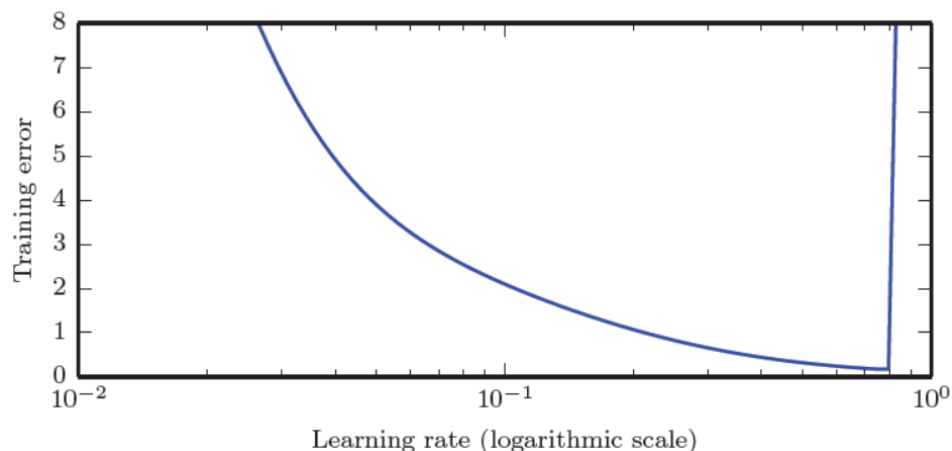
6.7 实践

- 1. 确定性能度量和目标
- 2. 建立一个合理的端到端的系统
- 3. 决定是否收集更多数据
 - 确定训练集上的性能是否可接受
 - 尝试增加更多的网络层或每层增加更多的隐藏单元，以增加模型的规模
 - 尝试调整学习率等超参数的措施来改进学习算法
 - 数据可能含太多噪声，或是可能不包含预测输出所需的正确输入，收集更干净的数据或是收集特征更丰富的数据集
 - 在测试集上性能差，收集更多的数据是最有效的解决方案之一，或者降低模型大小，或是改进正则化（调整超参数，如权重衰减系数，或是加入正则化策略，如Dropout）

6.7 实践

➤ 4. 选择超参数

- 手动选择：最小化受限于运行时间和内存预算的泛化误差，调整模型的有效容量以匹配任务的复杂性。有效容量受限于三个因素：模型的表示容量、学习算法成功最小化训练模型代价函数的能力以及代价函数和训练过程正则化模型的程度
- 当泛化误差以某个超参数为变量，作为函数绘制出来时，通常会表现为U形曲线。学习率可能是最重要的超参数
- 目标：提升测试集性能



6.7 实践

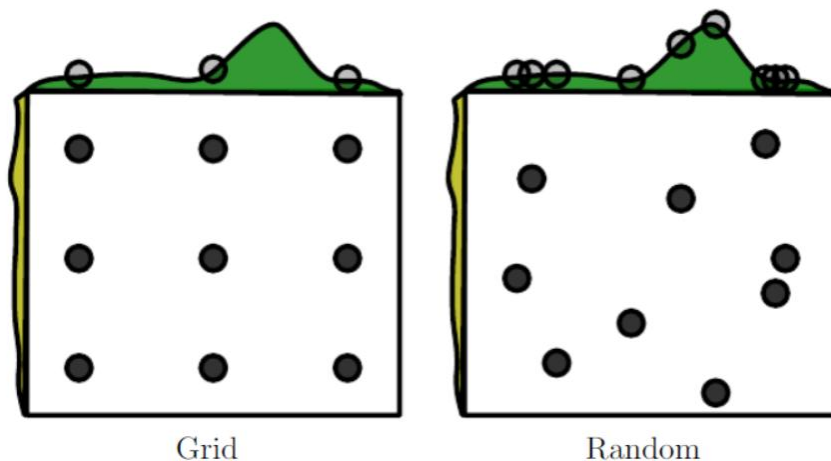
➤ 4. 选择超参数

- 例如由在相同类型的应用和架构上具有经验的人确定初始值，或者使用者在相似问题上具有几个月甚至几年的神经网络超参数调整经验，那么手动调整超参数能有很好的效果

6.7 实践

➤ 4. 选择超参数

- 自动超参数优化算法：只需要输入一个数据集，就可以输出学习的函数，而不需要手动调整超参数
- 搜索学习算法合适超参数，这其实是一种优化：寻找超参数来优化目标函数，例如验证误差，有时还会有一些约束（如训练时间，内存或识别时间的预算）
- 网格搜索：指数级增长计算代价
- 参数范围对数尺度



6.7 实践

➤ 4. 选择超参数

- 随机搜索：编程简单，使用更方便，能更快地收敛到超参数的良好取值：随机搜索(Bergstra and Bengio, 2012)
- 为每个超参数定义一个边缘分布，例如Bernoulli分布或范畴分布（分别对应着二元超参数或离散超参数），或者对数尺度上的均匀分布（对应着正实值超参数）
- 优点：不需要离散化超参数的值，当有几个超参数对性能度量没有显著影响时，随机搜索相比于网格搜索指数级地高效；没有浪费的实验，1个超参数的两个不同值，其他超参数取值也不同

6.7 实践

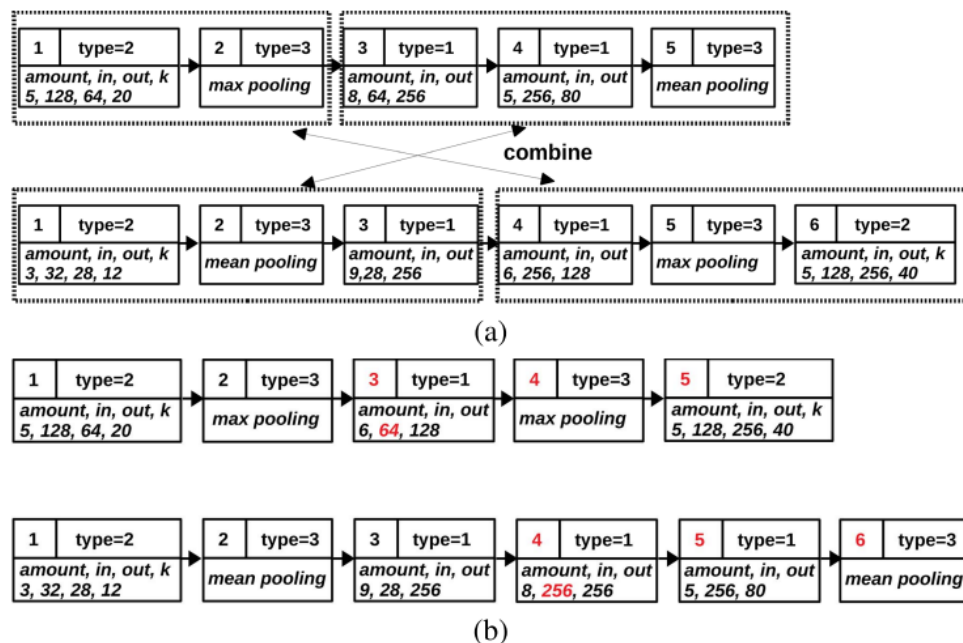
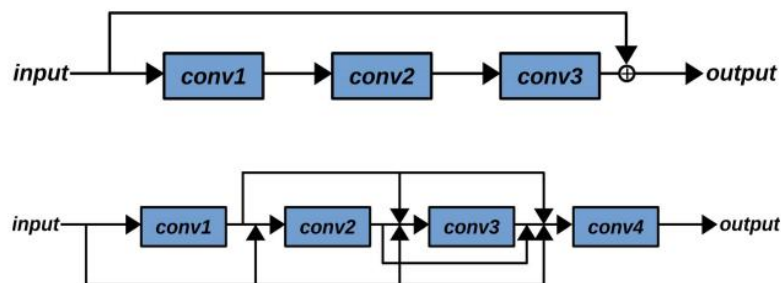
➤ 4. 选择超参数

- 基于模型的超参数优化
- 优化问题：决策变量是超参数，代价是超参数训练出来的模型在验证集上的误差
- 计算验证集上可导误差函数关于超参数的梯度，在大多数实际设定中，这个梯度是不可用的
- 弥补梯度的缺失，对验证集误差建模，然后通过优化该模型来提出新的超参数猜想
- 贝叶斯回归模型来估计每个超参数的验证集误差期望和该期望的不确定性

6.7 实践

➤ 4. 选择超参数

- 进化神经架构搜索
- 采用启发式方法，如遗传算法、粒子群优化等方法进行超参数搜索



6.7 实践：多位数字识别

➤ 1. 采集数据

- 街景车收集原始数据，然后操作员手动提供标签。
- 转录任务开始前有大量的数据处理工作，包括在转录前使用其他机器学习技术探测房屋号码。

➤ 2. 性能度量的选择和对这些度量的期望值

- 总原则是符合项目的业务目标，达到人类水平98%的准确率。
- 在保持准确率98% 的情况下，覆盖成了这个项目优化的主要性能度量。
- 随着卷积网络的改进，能够降低网络拒绝转录输入的置信度阈值，最终超出了覆盖95% 的目标。

6.7 实践：多位数字识别

➤ 3.快速建立一个合理的基准系统

- 对于视觉任务而言，基准系统是带有整流线性单元的卷积网络。转录项目开始于一个这样的模型，使用卷积网络输出预测序列
- 开始时使用一个尽可能简单的基准模型，该模型输出层包含 n 个不同的softmax 单元来预测 n 个字符的序列。使用与训练分类任务相同的方式来训练这些softmax 单元，独立地训练每个softmax 单元
- 反复细化这些基准，并测试每个变化是否都有改进

The End



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

Nankai University